

生产过程中的决策问题

摘要

本文建立了检测次数模型,根据次品率的假设值和置信水平,计算出最小的样本量;建立了成本最小优化模型,解决了生产过程中的决策问题;建立了多工序成本最小优化模型,解决了多工序情况下总成本最低的问题;建立了多约束成本最小优化模型,保证总成本在多条件约束下最小。

针对问题一,使用单侧检验设定关于抽样次品率的原假设与备择假设,抽样检测的次品数量服从二项分布,利用中心极限定理将二项分布近似为正态分布,得到抽样检测中次品数量的数学期望与方差,并结合正态分布的性质,可以得到抽样次品率的数学期望与方差,进而可知其正态分布,选用Z检验,检验抽样次品率与标称值是否相等,在检验过程中,将统计量标准化,移项得到所需的抽样样本量,使用右侧假设检验分别对拒收条件与接收条件进行具体分析,从而建立检测次数模型,通过绘制标准正态分布的密度函数图可以得到不同置信水平下的临界值,使用代入法代入模型得到具体检测次数,最终得到:在95%的信度下,需要的最少检测次数为98次;在90%的信度下,需要的最少检测次数为59次,并进行灵敏度分析,发现拒收临界值、抽样次品率和标称值对抽样检测次数影响较大,说明三者较为敏感,模型准确与可靠。

针对问题二,以总成本最小化为目标,以是否对零配件进行检测、是否对不合格品进行拆解等为决策变量,以成品次品率、总生产成本为约束条件,建立成本最小优化模型,找出最优决策组合,使用遗传算法(GA)通过初始化参数、定义适应度函数、创建遗传算法所需的基础组件、遍历每种情况求解,最终得到每种情况最优决策变量以及最小成本,其中情况4的总成本最小,为3362.0,使用遍历算法进行检验,发现求得的总成本大于使用遗传算法求解的总成本,可以确信遗传算法求解此模型较优,并进行灵敏度分析,发现次品率、调换损失和拆解费用对总成本影响甚微。

针对问题三,基于问题二,增加了多道工序和多个零配件,对给定的2道工序分别进行成本讨论,确定是否检测、是否拆解等5个决策变量,以最小化总成本为目标,以成品次品率、决策变量的取值为约束条件,建立多工序成本最小优化模型,使用混合整数线性规划算法(MILP)进行创建参数列表、初始化一个空列表、遍历列表中的每个情况、遍历结果列表等步骤求解模型,最终得到最小总成本为7544.0。

针对问题四,结合问题一的检测次数计算,更新了问题二和问题三中的检测成本公式,并新增了多个约束条件,即次品率、生产数量、检测成本等,建立了多约束成本最小优化模型,通过模拟退火算法进行初始化、定义目标函数等步骤求解,找到了不同决策变量组合下的成本最优解,得到结果:问题二中情况4的总成本最小,为3385.6;问题三中最小总成本为6572.0,灵敏度分析表明,零件次品率的变化对检测成本的影响较大,而成品次品率的变化则显著影响总成本,特别是在成品次品率较高时,调换和拆解成本急剧增加。

关键词:单侧假设检验;成本最小优化;遗传算法;混合整数线性规划算法;模拟退火算法



一、问题重述

1.1 问题的背景

某企业需要购买零配件 1 和零配件 2，将其装配，才能生产出畅销的电子产品。

1. 判断装配后的成品是否合格：

(1) 只要零配件 1 和零配件 2 其中一个不合格，则成品一定不合格。

(2) 零配件 1 和零配件 2 均合格的情况下，成品也不一定合格。

2. 对于不合格成品，企业的处理方法有：

(1) 选择报废。

(2) 对其进行拆解，拆解过程不会对零配件造成损坏，但需要花费拆解费用。

1.2 问题的重述

问题一：一批零配件（零配件 1 或零配件 2）的次品率不会超过某个标称值。企业想用抽样检测的方法来决定要不要接收这批零配件，检测费用企业自行承担。

1. 需要给企业设计一个检测次数尽量少的抽样检测方案，以节省检测费用。此方案便于企业在抽样检测后对零配件的次品率做出可靠判断。

2. 假设标称值是 10%，按照上述的抽样检测方案，在两种不同的置信水平下进行判断：

(1) 在 95% 的信度下，如果认定零配件次品率超过标称值，则拒收这批零配件。

(2) 在 90% 的信度下，如果认定零配件次品率不超过标称值，则接收这批零配件。

问题二：要求为企业生产过程中涉及的多个决策环节设计优化方案。生产过程各个阶段包括：

(1) 对零配件（1 和/或 2）是否进行检测。

(2) 对装配完成的成品是否进行检测。

(3) 对检测出的不合格成品是否进行拆解。

(4) 对退回的用户购买的不合格成品如何处理。

这些决策环节的目标是降低企业的总成本，同时确保企业在产品质量上的要求。每个决策都会涉及成本、收益以及次品率等因素，因此需要通过对生产成本进行单目标优化来设计具体的决策方案。

并解决以下问题：

1. 根据这些决策对表 1 中的各种情况提出具体的方案。

2. 解释决策的依据。

3. 提供相应的指标结果来支持决策。

问题三：给定具体值，即零配件、半成品和成品的次品率已知，要求重复问题 2，对 m 道工序、 n 个零配件，给出生产过程的决策方案。

问题四：要求通过抽样检测方法得到已知值零配件、半成品和成品的次品率是的前提下，重新分析和解决问题二和问题三，即再次得到最优决策组合和最小成本。



二、问题分析

这些问题之间存在着递进关系。问题一使用假设检验^[1]设计抽样检测方案，优化检测次数。问题二和问题三需要基于成本效益分析，问题二建立成本最小优化模型，处理单阶段的决策问题；问题三建立多工序成本最小优化模型，处理多工序情况下的决策问题，问题三是对问题二的复杂化。问题四将抽样检测应用于实际生产决策中，根据检测结果动态调整检测与拆解策略。这四个问题之间共同构成了一个完整的企业生产过程优化框架。

2.1 问题一的分析

采用单侧检验^[2]方法来设定原假设和备择假设，针对抽样次品率进行假设检验^[3]。由于次品数量服从二项分布^[4]，通过中心极限定理^[5]将其近似为正态分布，从而便于应用正态分布的性质进行计算。通过计算得到次品数量的期望和方差，进一步推导出次品率的期望和方差，并确认其符合正态分布。为了检验次品率是否超过标称值，选择使用 Z 检验^[6]，并在检验过程中对统计量进行了标准化处理。通过移项和计算，得到在不同置信水平下所需的最小抽样样本量，建立检测次数模型。

分别考虑了 95% 和 90% 的置信水平，并利用标准正态分布的密度函数图确定了相应的临界值。问题的结果可能是：在 95% 的置信水平下，可能需要进行 98 次检测；而在 90% 的置信水平下，可能所需的检测次数降低至 59 次。最后对模型中的拒收临界值、抽样次品率和标称值进行灵敏度分析。

2.2 问题二的分析

本题属于优化问题，关于如何通过决策变量来最小化生产过程中的总成本。通常涉及决策变量、目标函数、约束条件等要素。通过定义 7 个决策变量，以最小化生产总成本为目标函数，同时满足 2 个约束条件，建立成本最小优化模型。解决这类问题常用的方法有线性规划、整数规划、混合整数规划、启发式算法等。由于问题中包含整数变量和线性关系，本文问题适合使用混合整数线性规划 (MILP)^[7] 方法求解。通过遗传算法 (GA)^[8] 和遍历算法进行检验，并对不同决策变量进行灵敏度分析。

2.3 问题三的分析

本题属于典型的生产优化问题。解决这类问题常用的方法包括线性规划、整数规划、混合整数线性规划、启发式算法以及动态规划等。本文问题适合使用混合整数线性规划 (MILP) 方法，因为它涉及离散决策变量和连续成本变量。定义决策变量、以最小化总成本为目标函数、设置约束条件（如成品次品率、决策变量的取值），建立多工序成本最小优化模型。求解这类模型常用的方法包括混合整数线性规划算法、分支定界法、割平面法等。本题适用混合整数线性规划算法求解。通过创建参数列表、初始化空列表、遍历列表中的每个情况、遍历结果列表等步骤，最终得到最小总成本。

2.4 问题四的分析

本题属于生产优化中的成本最小化问题，这类问题通常通过优化算法来解决。常用的方法包括线性规划、整数规划、混合整数规划、模拟退火算法等。本文问题使用模拟退火算法，因为它能够处理复杂的约束条件和决策变量组合。建模过程包括定义决策变量、建立目标函数、设定约束条件（如次品率、生产数量、检



测成本等)，并构建成本最小优化模型。求解这类模型常用的方法有模拟退火、遗传算法、线性规划等。本题使用模拟退火算法求解，通过初始化、定义目标函数等步骤，找到了不同决策变量组合下的成本最优解，最终求得问题二与问题三的最小成本。

三、模型假设

1. 假设抽样的零配件能够代表整批次的品质。
2. 假设检测方法能够准确判断零配件是否合格。
3. 假设零配件的质量相互独立，一个零配件的质量不影响另一个。
4. 假设市场对产品的需求是无限的，所有成品都能以固定价格售出。

四、符号说明

表 4.1 符号说明表

符号	单位	说明
p_0	%	标称值
$\hat{p}$	%	抽样次品率
n	个	抽样样本量
X	个	样品检测中次品数量
$E(\hat{p})$	/	抽样次品率的数学期望
$V(\hat{p})$	/	抽样次品率的方差
d	%	抽样次品率与标称值的误差值
Z_α	/	拒收临界值
Z_β	/	接收临界值
D_i	/	零配件 i 的决策变量
D_f	/	成品的决策变量
D_a	/	不合格成品的决策变量
D_r	/	退回的成品的决策变量
N	个	计划生产合格的成品数量
$p_i (i = 1, 2, \dots, n)$	%	次品率

五、模型建立与求解

5.1 问题一的模型建立与求解

采用单侧检验^[4]，以二项分布为基础，通过中心极限定理转化为正态分布，计算抽样次品率的期望和方差，使用 Z 检验比较次品率与标称值，通过标准化和移项确定所需样本量。结合置信水平分析拒收和接收条件，建立检测次数模型，并通过标准正态分布的密度函数图确定临界值，最终得到具体检测次数，并进行灵敏度分析。

5.1.1 模型建立

本题是关于比例的假设检验问题，企业通常只关心次品率是否“超过”标称



值, 而不是“小于或等于”标称值, 因此这是一个单侧检验。

假设 p_0 为标称值, $\hat{p}$ 为抽样次品率, n 为抽样样本量, X 为样品检测中次品数量。

其中, 对于抽样次品率有:

$$\hat{p} = \frac{X}{n} \quad (5-1)$$

首先设定假设, 原假设 H_0 为抽样次品率 $\hat{p}$ 不大于标称值 p_0 , 即 $\hat{p} \leq p_0$; 备择假设 H_1 为抽样次品率 $\hat{p}$ 大于标称值 p_0 , 即 $\hat{p} > p_0$ 。

由于每次抽取的零配件只有非次品和次品两种结果, 因此抽样检测的次品数量 X 服从二项分布 $X \sim B(n, \hat{p})$ 。根据中心极限定理, 当样本量 $n \geq 30$ 时, 二项分布将近似于正态分布 $X \sim N(n\hat{p}, n\hat{p}(1-\hat{p}))$ 。因此, 可以知道次品数量 X 的数学期望为 $n\hat{p}$, 方差为 $n\hat{p}(1-\hat{p})$ 。

根据正态分布的性质, 结合上述次品数量 X 的数学期望与方差, 可以知道抽样次品率 $\hat{p}$ 的数学期望为:

$$E(\hat{p}) = E\left(\frac{X}{n}\right) = \frac{E(X)}{n} = \frac{n\hat{p}}{n} = \hat{p} \quad (5-2)$$

抽样次品率 $\hat{p}$ 的方差为:

$$V(\hat{p}) = V\left(\frac{X}{n}\right) = \frac{V(X)}{n^2} = \frac{n\hat{p}(1-\hat{p})}{n^2} = \frac{\hat{p}(1-\hat{p})}{n} \quad (5-3)$$

由于抽样次品率 $\hat{p}$ 是对标称值 p_0 的估计, 并且我们假设样本量足够大, 那么抽样次品率应该接近标称值。

因此, 可以得到抽样次品率 $\hat{p}$ 服从正态分布 $\hat{p} \sim N(p_0, np_0(1-p_0))$ 。

为了验证抽样次品率 $\hat{p}$ 与标称值 p_0 是否相等, 我们选用 Z 检验。

需要判断从样品中估计得到的次品率 $\hat{p}$ 与标称值 p_0 之间是否存在显著差异, 我们引入 Z 统计量, 对 Z 统计量标准化有:

$$Z = \frac{\hat{p} - p_0}{\sqrt{\frac{p_0(1-p_0)}{n}}} \quad (5-4)$$

移项得到所需的抽样样本量 n 为:

$$n = \frac{p_0(1-p_0)Z^2}{(\hat{p} - p_0)^2} = \frac{p_0(1-p_0)Z^2}{d^2} \quad (5-5)$$

式 5-5 中, d 表示抽样次品率 $\hat{p}$ 与标称值 p_0 的误差值。

针对题中所给两种情形, 首先假设拒收临界值为 Z_α , 假设接收临界值为 Z_β 。根据具体的置信水平结合标准正态分布表可以得到具体数值。

由于备择假设 H_1 为 $\hat{p} > p_0$, 因此拒绝域位于右侧, 所以选择使用右侧假设检验。

在检验统计量落在拒绝域内, 拒绝原假设时, 拒收条件才成立。拒收条件下右侧检验标准正太分布图如下图 5.1 所示:



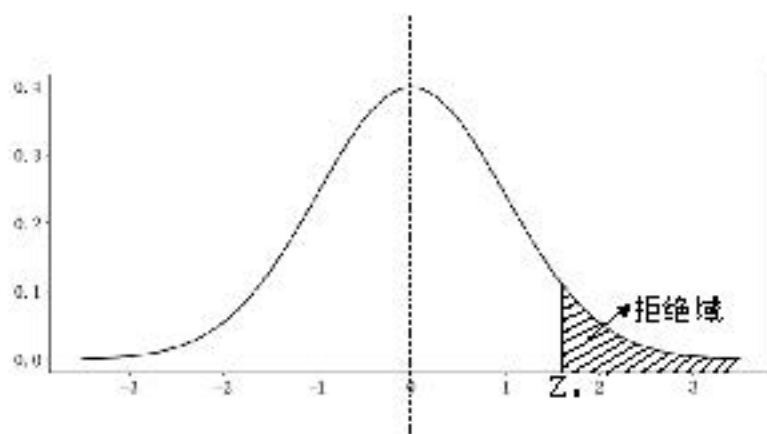


图 5.1 拒收条件下右侧检验图

则选择备择假设 H_1 ，为 $\hat{p} > p_0$ 。

因此，可以确定拒收条件为：

$$\frac{\hat{p} - p_0}{\sqrt{\frac{p_0(1-p_0)}{n}}} > Z_\alpha \quad (5-6)$$

化简得到拒收条件下抽样次品率 $\hat{p}$ 的取值范围为：

$$\hat{p} > p_0 + Z_\alpha \sqrt{\frac{p_0(1-p_0)}{n}} \quad (5-7)$$

对上式 5-7 移项并结合上式 5-1 可以进一步得到拒收条件下样品检测中次品数量 X 的取值范围为：

$$X > np_0 + Z_\alpha \sqrt{np_0(1-p_0)} \quad (5-8)$$

那么，可以确定拒收条件下抽样样本量 n 的范围为：

$$n > \frac{p_0(1-p_0)Z_\alpha^2}{(\hat{p} - p_0)^2} \quad (5-9)$$

在检验统计量落在接收域内，接收原假设时，接收条件才成立。接收条件下右侧检验标准正太分布图如下图 5.2 所示：

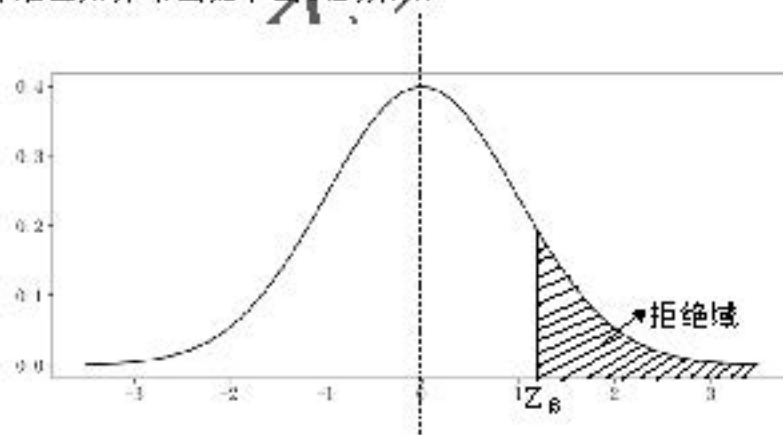


图 5.2 接收条件下右侧检验图

则选择原假设 H_0 ，为 $\hat{p} \leq p_0$ 。

因此，可以确定接收条件为：



$$\frac{\hat{p} - p_0}{\sqrt{\frac{p_0(1-p_0)}{n}}} < Z_\beta \quad (5-10)$$

化简得到抽样次品率 $\hat{p}$ 的接收范围为:

$$\hat{p} < p_0 + Z_\beta \sqrt{\frac{p_0(1-p_0)}{n}} \quad (5-11)$$

则可以进一步得到样品检测中次品数量 X 的接收范围为:

$$X < np_0 + Z_\beta \sqrt{np_0(1-p_0)} \quad (5-12)$$

那么, 可以确定接收条件下抽样样本量 n 的范围为:

$$n < \frac{p_0(1-p_0)Z_\beta^2}{(\hat{p} - p_0)^2} \quad (5-13)$$

综上所述, 可以建立检测次数模型:

$$n = \frac{p_0(1-p_0)Z^2}{d^2} \quad (5-14)$$

5.1.2 模型求解

题中给定了两种置信情况, 即 95% 和 90% 的信度, 依次求在这两种信度下的标准正态分布临界值。

通过借助 python 工具绘制标准正态分布的密度函数图如下图 5.3 所示, 可以得到不同置信水平下的临界值。

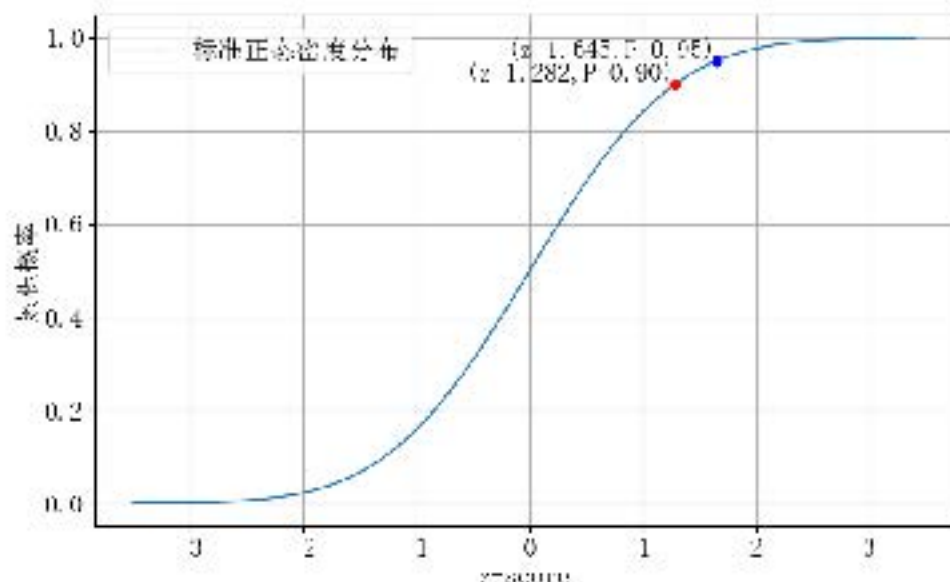


图 5.3 标准正态分布的密度函数图

据上图 5.3 可知, 95% 置信水平下的标准正态分布的临界值 Z_α 为 1.645; 90% 置信水平下的标准正态分布的临界值 Z_β 为 1.282。

根据题意可知标称值 p_0 为 10%, 假设抽样次品率 $\hat{p}$ 为 15%, 在拒收条件下的临界值 Z_α 为 1.645, 在接收条件下的临界值 Z_β 为 1.282。使用代入法分别代入上式模型 5-14 求解得到拒收条件下所需最少检测次数 $n_1 = 98$; 接收条件下所需



最少检测次数 $n_2 = 59$ 。

5.1.3 问题的结论

对于情形一，在 95% 的信度下，需要的最少检测次数为 $n_1 = 98$ 。

对于情形二，在 90% 的信度下，需要的最少检测次数为 $n_2 = 59$ 。

5.1.4 检验分析

为了评估模型的稳健性，我们对多个变量进行灵敏度分析，包括拒收临界值 Z_α 、抽样次品率 $\hat{p}$ 和标称值 p_0 。

以拒收条件为例，接收条件的灵敏度分析与拒收条件同理。

1. 拒收临界值的灵敏度分析

保持标称值 p_0 为 10%、检测误差 d 为 5%，控制拒收临界值 Z_α 的波动在 $[1.282, 2.326]$ 范围内， Z_α 的变化对抽样样本量 n 的影响如下图 5.4 所示：

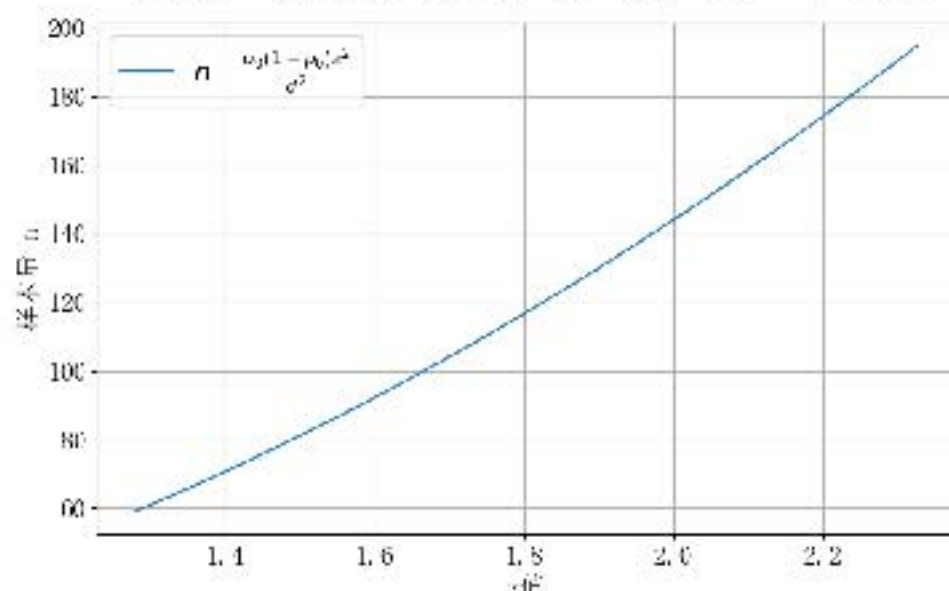


图 5.4 拒收临界值的变化对抽样样本量的影响图

观察上图 5.4 可知，在 Z_α 值较小的区间，样本量较低，随着拒收临界值 Z_α 的增加，抽样样本量 n 呈现上升趋势，反映了拒收临界值与样本量之间呈正相关关系。

而拒收临界值 Z_α 对应不同的置信水平，说明如果企业仅要求 90% 的置信水平，样本量的要求较低，如果企业要求更高的置信度，如 99%，则样本量的需求显著增加。

如果企业希望提高对次品率检测的置信水平，则需要增大样本量，而样本量的增加是为了在更高的置信水平下保持检测的精确性，减少次品率判断中的误差。

2. 抽样次品率的灵敏度分析

保持标称值 p_0 为 10%、拒收临界值 Z_α 为 1.645，控制抽样次品率 $\hat{p}$ 的波动在 $[0, 0.05) \cup (0.15, 0.3]$ 范围内， $\hat{p}$ 的变化对抽样样本量 n 的影响如下图 5.5 所示：



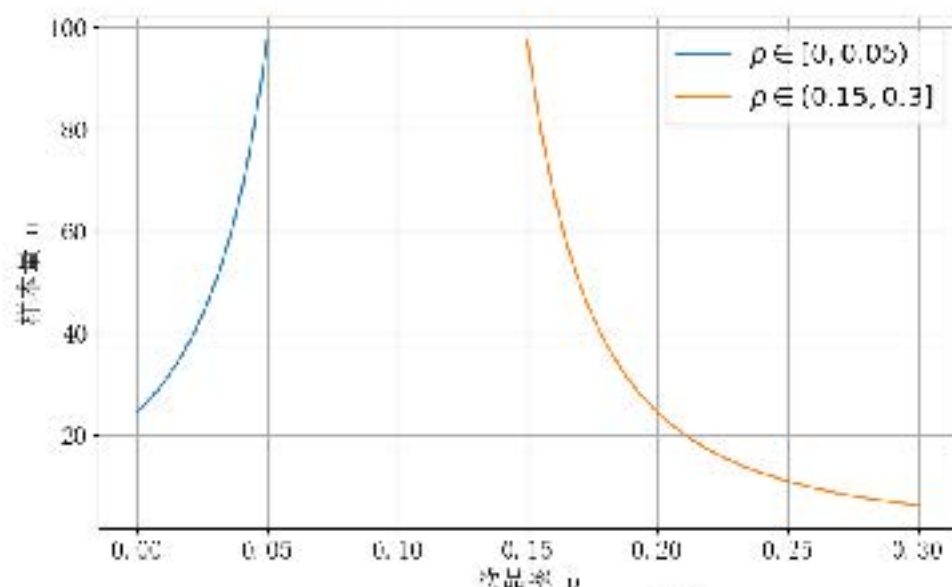


图 5.5 抽样次品率的变化对抽样样本量的影响图

观察上图 5.5 可知，对于左侧曲线，随着 $\hat{p}$ 接近 0.05，所需的抽样样本量 n 急剧上升，因为 $\hat{p}$ 接近标称值 p_0 ，次品率之间的差异较小，为了区分出这微小的差异，所需的样本量非常大；随着 $\hat{p}$ 接近于 0，抽样样本量 n 逐渐减少，因为较低的抽样次品率 $\hat{p}$ 远低于标称值 p_0 ，差异明显，所需的样本量相对较小即可做出拒收判断。对于右侧曲线，在 $\hat{p}$ 远大于 p_0 时，样本量呈下降趋势，因为随着抽样次品率与标称值的差距增大，能够轻松地区分出抽样次品率的高低，样本量的需求减少，尤其是在 $\hat{p}$ 逐渐接近 0.3 时，抽样次品率差异明显，样本量需求显著降低。

分析可知，对于小范围次品率波动，即 $\hat{p}$ 接近标称值 p_0 时，为了区分小的抽样次品率差异，所需的样本量迅速上升，这表明当实际次品率接近于标称值时，我们需要非常大的样本量才能作出有效判断；对于远离标称值时，当抽样次品率明显低于或高于标称值时，样本量要求较小，因为此时差异较大，可以通过较少的样本量来判断抽样次品率是否超出标称值。

说明抽样次品率接近标称值时，样本量敏感性高；抽样次品率远离标称值时，样本量需求减少。

3. 标称值的灵敏度分析

保持拒收临界值 c_a 为 1.645，抽样次品率 $\hat{p}$ 为 11%，控制标称值 p_0 的波动在 $[0.05, 0.09] \cup [0.13, 0.15]$ 范围内， p_0 的变化对抽样样本量 n 的影响如下图 5.6 所示：



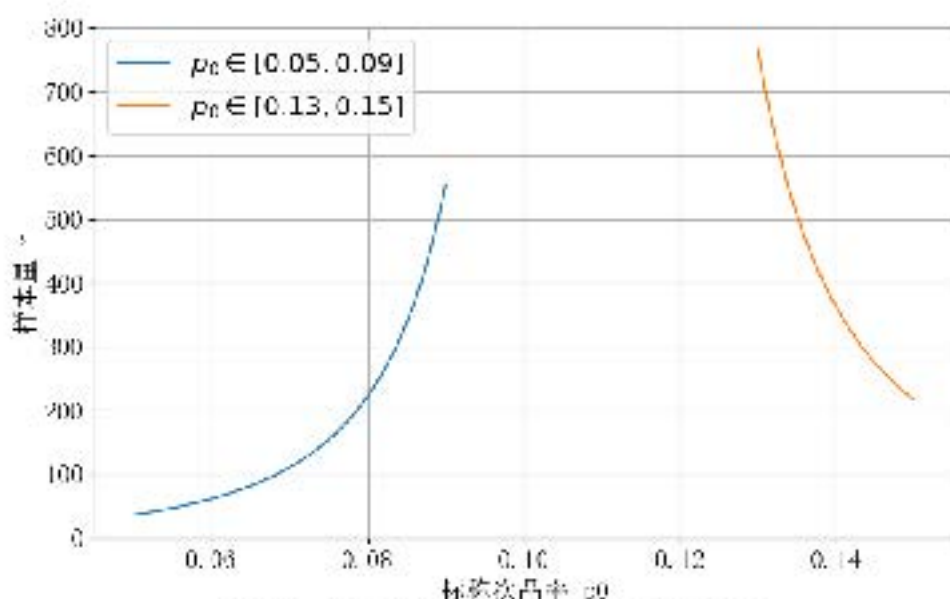


图 5.6 标称值的变化对抽样样本量的影响图

观察上图 5.6 可知，对于左侧曲线，当 p_0 接近 0.09 时，样本量显著增加，因为抽样次品率 $\hat{p}$ 与 p_0 之间的差距较小，较难区分实际次品率 $\hat{p}$ 与标称值的差异，需要较大的样本量来保证检验的准确性；当 p_0 越小，如接近 0.05 时，样本量下降，表明，标称值远低于抽样次品率 $\hat{p}$ 时，差异较为显著，不需要大量样本就可以做出判断。对于右侧曲线，当 p_0 接近 0.15 时，样本量减少，因为标称值远高于实际抽样次品率 $\hat{p}$ ，可以很容易判断成品质量问题，不需要大量样本；当 p_0 接近 0.13 时，样本量略有增加，标称值和实际抽样次品率 $\hat{p}$ 之间的差距变小，因此需要更多样本来准确区分。

分析可知，标称值与抽样次品率 $\hat{p}$ 的差异越小，所需样本量越大，当 p_0 接近 $\hat{p}$ 时，无论标称次品率是低于还是高于 $\hat{p}$ 值，所需的样本量都显著增加，表明当实际次品率和标称次品率差异不大时，需要更多的样本来进行区分和判定；标称值远离抽样次品率 $\hat{p}$ 时，所需样本量较少，无论标称值是远高于还是低于 $\hat{p}$ ，当二者差距较大时，所需样本量较少，因为差异明显，不需要大量样本就可以得出结论。

5.1.5 小结

通过单侧检验和中心极限定理，将次品数量的二项分布近似为正态分布，计算其期望和方差，以 Z 检验评估次品率与标称值差异，确定样本量并分析拒收与接收条件，建立检测次数模型，并通过标准正态分布图确定临界值，得出具体检测次数，并进行灵敏度分析。

5.2 问题二的模型建立与求解

本问以最小化总生产成本为目标，定义 7 个决策变量，并考虑成品次品率约束和总生产成本约束 2 个约束条件，建立成本最小优化模型，选择使用遗传算法 (GA) 求解模型，找出最优决策变量的组合，最终可以得到每种情况的决策变量值和对应的最小总成本，使用遍历算法检验，并对 3 个变量进行灵敏度分析。



5.2.1 模型建立

本题对总生产成本进行单目标优化,使企业在生产过程中产生的成本最低。

1. 决策变量的确定

根据题目要求,要使生产成本尽可能低,需要 7 个决策变量:

(1) 是否对零配件 1 进行检测,假设零配件 1 为 D_1 。

(2) 是否对零配件 2 进行检测,假设零配件 2 为 D_2 。

(3) 是否对成品进行检测,假设成品为 D_f 。

上述三个决策变量的取值为 $D_i \in \{0,1\}$ ($i=1,2,f$),当取值为 0 时,表示对其不进行检测;当取值为 1 时,表示对其进行检测。

(4) 是否对不合格成品进行拆解,假设不合格成品为 D_g ,取值为 $D_g \in \{0,1\}$,当取值为 0 时,表示对其不进行拆解;当取值为 1 时,表示对其进行拆解。

(5) 是否对退回的成品进行拆解,假设退回的成品为 D_r ,取值为 $D_r \in \{0,1\}$,当取值为 0 时,表示直接丢弃不合格成品;当取值为 1 时,表示对其进行拆解。

(6) 是否对拆解后的零配件进行检测,假设拆解后的零配件为 D_{gi} ($i=1,2$),取值为 $D_{gi} \in \{0,1\}$,当取值为 0 时,不对其进行检测,直接与其他零配件进行重新装配;当取值为 1 时,对其进行检测,如果合格,再进行装配,如果不合格,则丢弃。

(7) 是否对退回成品拆解后的零配件进行检测,假设退回成品拆解后的零配件为 D_{ri} ($i=1,2$),取值为 $D_{ri} \in \{0,1\}$,当取值为 0 时,直接重新装配;当取值为 1 时,对其进行检测。

对于题中所提到的重复步骤,我们只考虑最多对零配件进行二次装配。

2. 目标函数分析

为了对每个阶段的决策进行优化,需要最小化总成本,而总成本由零配件采购成本、检测成本、装配成本、拆解成本、调换损失成本组成,对各个成本进行分析:

假设计划生产的成品数量为 N 。

(1) 零配件采购成本

由于零配件存在次品率,所以需要采购足够数量的零配件,确保在丢弃不合格零配件后,仍有足够的合格零配件用于生产,零配件采购成本与次品率检测结果相关。

假设 c_{di} 为第 i 个零配件购买单价,则零配件采购成本 C_{dw} 为:

$$C_{dw} = N \sum_{i=1}^2 c_{di} \quad (5-15)$$

(2) 检测成本

检测成本包括对零配件 1、零配件 2、成品、拆解后的零配件和调换退回的产品进行检测。

假设 c_{di} 为每个零配件 i 的检测成本, c_g 为每个成品检测成本, c_r 为每个不合格成品的拆解成本, p_1 为零配件 1 的次品率, p_2 为零配件 2 的次品率, p_f 为成品次品率, p_f' 为两个零配件装配成成品的次品率,则 p_f' 与各个零配件次品率之间关系为:



$$p_f' = 1 - (1 - p_1)(1 - p_2) \quad (5-16)$$

式中, $(1 - p_1)(1 - p_2)$ 表示零配件 1 与零配件 2 都合格, 在两个零配件都合格的情况下成品才合格。

假设 p_i ($i=1,2$) 为零配件的次品率, D_i ($i=1,2$) 为是否对零配件进行检测的决策变量, 则总的检测成本 C_{dc} 为:

$$C_{dc} = N \left[\sum_{i=1}^2 D_i c_{di} + D_f c_{df} + \left[D_i (1 - p_i) N p_i + (1 - D_i) N (1 - (1 - p_i')(1 - p_i)) \right] \right. \\ \left. \left(\sum_{i=1}^2 D_i D_i c_{di} (D_{di} + D_{di} (1 - D_i)) \right) \right] \quad (5-17)$$

式中, 如果对检测出的不合格成品以及调换退回的不合格成品进行拆解, 有可能进行检测, 也有可能不进行检测, 直接与其他零配件再装配, $N \sum_{i=1}^2 D_i c_{di}$ 表示零配件的初始检测成本; $N D_f c_{df}$ 表示对成品的检测成本; $(1 - p_i')(1 - p_i)$ 表示检测的不合格率, 当零配件检测合格时才进行装配, 企业可以根据具体的成本和次品率来决定是否进行检测。

(3) 装配成本

两种零配件组装成成品时会产生装配成本, 包括初始时的装配成本、拆解和调后再重组的装配成本, 装配成本只与生产成品数量有关, 与次品率、检测结果无关。

假设 c_a 为每件成品的装配成本, N 为需要装配的某一个零配件的数量。

则需要装配的数量 N' 与零配件购买数量 N 的关系为

$$N' = [D_i (1 - p_i) N + (1 - D_i) N] \quad (5-18)$$

式中, $D_i (1 - p_i) N$ 表示对零配件进行检测, 并且检测合格的零配件装配的成本, $(1 - D_i) N$ 表示不对零配件进行检测, 直接装配所有零配件的成本。

因此, 分为以下两种情况对装配成本进行讨论, 即装配之前是否对零配件进行检测, 对于不合格的成品以及调换退回的成品选择是否对其进行拆解, 如果拆解, 再选择是否对拆解后的零配件进行检测, 如果不拆解, 则直接丢弃。

首先考虑对零配件进行检测的情况, 对成品进行拆解后, 直接对零配件进行装配的成本 C_1 为:

$$C_1 = D_i (1 - p_i) N \left[\sum_{i=1}^2 D_i D_i p_i (1 - D_{di}) \frac{c_a}{2} \right] \quad (5-19)$$

拆解成品后, 对拆解得到的零配件进行检测, 再装配的成本 C_2 为:

$$C_2 = D_i (1 - p_i) N \left[\sum_{i=1}^2 D_i D_i p_i D_{di} (1 - p_i) \frac{c_a}{2} (1 - D_i) \right] \quad (5-20)$$

对调换退回的成品进行拆解后, 不进行检测, 直接装配的成本 C_3 为:

$$C_3 = D_i (1 - p_i) N \left[\sum_{i=1}^2 (1 - D_i) D_i p_i (1 - D_{di}) \frac{c_a}{2} \right] \quad (5-21)$$

对调换退回的成品进行拆解并进行检测, 合格零配件再进行装配的成本 C_4 为:



$$C_4 = D_i(1-p_i)N \left[\sum_{j=1}^2 (1-D_f)D_r p_f D_n(1-p_i) \frac{c_a}{2} (1-D_i) \right] \quad (5-22)$$

其次考虑不对零配件进行检测的情况，拆解后直接进行装配的成本 C_5 为：

$$C_5 = (1-D_i)N \left[\sum_{j=1}^2 D_f D_n [1-(1-p_f')(1-p_f)] (1-D_n) \frac{c_a}{2} \right] \quad (5-23)$$

拆解后对零配件进行检测后再装配的成本 C_6 为：

$$C_6 = (1-D_i)N \left[\sum_{j=1}^2 D_f D_n [1-(1-p_f')(1-p_f)] D_n(1-p_i) \frac{c_a}{2} (1-D_i) \right] \quad (5-24)$$

对调换退回的成品进行拆解后直接装配的成本 C_7 为：

$$C_7 = (1-D_i)N \left[\sum_{j=1}^2 (1-D_f)D_r [1-(1-p_f')(1-p_f)] (1-D_n) \frac{c_a}{2} \right] \quad (5-25)$$

对调换退回的成品进行拆解后，再进行检测，检测到的合格品进行装配的成本 C_8 为：

$$C_8 = (1-D_i)N \left[\sum_{j=1}^2 (1-D_f)D_r [1-(1-p_f')(1-p_f)] D_n(1-p_i) \frac{c_a}{2} (1-D_i) \right] \quad (5-26)$$

那么，总的装配成本 $C_{\text{总}}$ 为：

$$C_{\text{总}} = Nc_0 + C_1 + C_2 + C_3 + C_4 + C_5 + C_6 + C_7 + C_8 \quad (5-27)$$

式中， Nc_0 表示初始装配费用。

虽然装配成本是每件成品的固定费用，但是装配后的成品可能会因为零配件的质量问题而不合格，因此，企业可能需要重新装配不合格成品的零配件，如果成品不合格而不进行拆解重新使用零配件，那么这部分不合格品的装配成本是浪费的，然而，从装配成本的计算本身来看，通常不对这部分做额外处理，因为无论成品是否合格，装配环节已经完成，装配成本已发生。

(4) 拆解成本

拆解成本包括不合格成品和调换退回成品的拆解费用，通过拆解，企业可以回收零配件并重新利用，从而减少零配件的再次采购成本。

总的拆解成本 $C_{\text{拆}}$ 为：

$$C_{\text{拆}} = D_i(1-p_i)Nc_r(D_n + D_r)p_f + (1-D_i)Nc_r(D_n + D_r)[1-(1-p_f')(1-p_f)] \quad (5-28)$$

式中，对于 $D_i(1-p_i)Nc_r(D_n + D_r)p_f$ ，表示检测后的不合格成品的拆解费用， $(1-D_i)Nc_r(D_n + D_r)[1-(1-p_f')(1-p_f)]$ 表示调换退回成品的拆解费用。

(5) 调换损失成本

对于进入市场的不合格成品，产生调换损失，并选择是否对退回的不合格成品进行拆解，若拆解，再选择是否对其进行检测。

这个成本不仅包括产品本身的调换费用，还可能包括物流、处理退货的人工成本以及企业信誉受损带来的间接损失。

假设 c_l 为不合格成品的调换损失。

则总的调换损失成本为：



$$C_{\text{ex}} = D_i(1-p_i)Np_f[(1-D_f)(c_i+s)] \\ + (1-D_i)N[1-(1-p_f')(1-p_f)][(1-D_f)(c_i+s)] \quad (5-29)$$

式中， s 表示成品市场售价。

综上所述，目的是协调减低各个阶段的成本，从而使总成本最低。据此本文确立如下 1 个优化目标：

$$\min C_{\text{total}} = C_{\text{buy}} + C_{\text{dm}} + C_{\text{m}} + C_{\text{diz}} + C_{\text{ex}} \quad (5-30)$$

3. 约束条件分析

根据分析题意可知，有 2 个约束条件，分别为：

(1) 成品次品率约束

企业通常会设定一个次品率的阈值，并采取各种质量控制措施，确保次品率控制在允许的范围内，以维护企业的正常运营和社会责任。因此，成品次品率需要满足约束条件：

$$p_f \leq p_{\text{max}} \quad (5-31)$$

(2) 总生产成本约束

由于总生产成本不得超过企业预算，因此，需要满足约束条件：

$$C_{\text{total}} < B \quad (5-32)$$

综上所述，可以建立成本最小优化模型：

$$\min C_{\text{total}} = C_{\text{buy}} + C_{\text{dm}} + C_{\text{m}} + C_{\text{diz}} + C_{\text{ex}} \\ s.t. \begin{cases} p_f \leq p_{\text{max}} \\ C_{\text{total}} < B \end{cases} \quad (5-33)$$

5.2.2 模型求解

假设计划生产成品数量 $N=100$ 。

已知列表数据为题中所给出的表 1，做调查表格式，得到下表 5.1 多情况参数表：

表 5.1 多情况参数表

情况	p_i	c_{d1}	c_{d1}	p_i	c_{d2}	c_{d2}	p_f	c_a	c_g	c_i	c_i
1	10%	4	2	10%	18	3	10%	6	3	6	5
2	20%	4	2	20%	18	3	20%	6	3	6	5
3	10%	4	2	10%	18	3	10%	6	3	30	5
4	20%	4	1	20%	18	1	20%	6	2	30	5
5	10%	4	8	20%	18	1	10%	6	2	10	5
6	5%	4	2	5%	18	3	5%	6	3	10	40

由于涉及到多个决策变量和约束条件，形成了一个复杂的决策空间，而决策变量是离散的，因此，我们考虑使用遗传算法（GA）求解模型。

遗传算法具有全局搜索能力，通过模拟自然选择和遗传机制^[10]，能够在复杂问题的搜索空间中找到可能的最优解。

使用遗传算法检验的步骤如下：

(1) 初始化参数，设定种群大小 50、交叉概率 0.5、变异概率 0.2 以及最大迭代代数 40。

(2) 定义适应度函数，该函数根据个体（决策变量组合）和特定情况下的参数计算总成本，并将其作为个体的适应度。



(3) 创建遗传算法所需的基础组件。首先定义适应度类，用于评估个体的优劣，并设置为最小化目标。接着定义个体类，表示算法中的一个解，包含一组决策变量。最后初始化工具箱，包含用于生成个体和种群的函数，以及定义遗传操作的函数。

(4) 遍历每种情况。首先更新适应度函数，使其适用于当前情况。再初始化种群，即创建一组随机的个体作为算法的起始点。最后运行遗传算法：使用选择操作从当前种群中选择个体进行繁殖、使用交叉操作对选中的个体进行配对并交换部分基因、使用变异操作对个体进行随机改变以增加种群的多样性。紧接着评估种群中每个个体的适应度，保留当前代中最好的个体。重复以上步骤直到达到最大迭代代数。

(5) 输出结果。对于每种情况，输出最优个体的决策变量组合和对应的总成本。

5.2.3 问题的结论

使用遗传算法求解得出各个情况的决策方案以及总成本，如下表 5.2 所示：

表 5.2 遗传算法求解的不同情况决策变量值和对应的最小总成本表

情况	D_1	D_2	D_f	D_d	D_r	D_{c1}	D_{c2}	D_{r1}	D_{r2}	C_{total}
1	0	0	1	0	0	0	1	0	0	3403.0
2	0	0	1	0	0	1	1	0	1	3403.0
3	0	0	1	0	0	1	1	1	0	3403.0
4	1	1	1	0	0	1	1	0	1	3362.0
5	0	1	1	0	0	1	0	0	1	3382.0
6	0	0	1	0	0	0	1	0	0	3403.0

对于情况 1，当决策变量 D_f 、 D_{c2} 取值为 1，其他决策变量取值为 0 时，即选择对成品和拆解后的零配件 2 进行检测时，总成本最小，为 3403.0。

对于情况 2，当决策变量 D_f 、 D_{c1} 、 D_{c2} 、 D_{r2} 取值为 1，其他决策变量取值为 0 时，即选择对成品、拆解后的零配件 1 和 2、退回成品拆解后的零配件 2 进行检测时，总成本最小，为 3403.0。

对于情况 3，当决策变量 D_f 、 D_{c1} 、 D_{c2} 、 D_{r1} 取值为 1，其他决策变量取值为 0 时，即选择对成品、拆解后的零配件 1 和 2、退回成品拆解后的零配件 1 进行检测时，总成本最小，为 3403.0。

对于情况 4，当决策变量 D_1 、 D_2 、 D_f 、 D_{c1} 、 D_{c2} 、 D_{r2} 取值为 1，其他决策变量取值为 0 时，即选择对零配件 1 和 2、成品、拆解后的零配件 1 和 2、退回成品拆解后的零配件 2 进行检测时，总成本最小，为 3362.0。

对于情况 5，当决策变量 D_2 、 D_f 、 D_{c1} 、 D_{r2} 取值为 1，其他决策变量取值为 0 时，即选择对零配件 2、成品、拆解后的零配件 1 和退回成品拆解后的零配件 2 进行检测时，总成本最小，为 3382.0。

对于情况 6，当决策变量 D_f 、 D_{c2} 取值为 1，其他决策变量取值为 0 时，即选择对成品和拆解后的零配件 2 进行检测时，总成本最小，为 3403.0。

5.2.4 检验分析

1. 遍历算法检验

为了确保结果的准确性，找到问题的所有可行解决方案，我们使用遍历算法检验，系统地检查所有的可能组合，确保找到问题的所有可行解决方案。



将遍历算法与遗传算法进行比较,可以确保求解得准确性.对于每个决策点,遍历算法会考虑所有可能的组合,并计算每种组合的总成本.

使用遍历算法求解的步骤如下:

(1) 定义一个函数,用于计算每种情况的总成本.该函数接受决策变量和上表 5.1 参数作为输入,并计算各个成本:基于单位数量和决策变量的值计算零配件购买成本;基于决策变量和参数计算检测成本、装配成本、拆解成本、调换损失成本.函数最后输出总成本以及相关的数量.

(2) 遍历参数列表中的每个情况.先初始化最小成本为无穷大,用于存储当前情况的最小成本.初始化最佳决策变量组合和数量为 None,用于存储导致最小成本的决策变量组合和数量.再使用迭代工具生成所有可能的决策变量组合.对于每个决策变量组合,调用成本计算函数,计算总成本和数量.如果计算得到的成本小于当前记录的最小成本,则更新最小成本,并记录当前决策变量组合和数量.在完成所有决策变量组合的遍历后,输出当前情况的最小成本和对应的最佳决策变量组合及数量.输出结果如下表 5.3 所示:

表 5.3 遍历算法求解的不同情况决策变量值和对应的最小总成本表

情况	D_1	D_2	D_f	D_d	D_r	D_{c1}	D_{c2}	D_{r1}	D_{r2}	C_{total}
1	0	0	1	0	0	0	0	0	0	3700.0
2	0	0	1	0	0	0	0	0	0	3700.0
3	0	0	1	0	0	0	0	0	0	3700.0
4	1	1	1	0	0	0	0	0	0	3560.0
5	0	1	1	0	0	0	0	0	0	3580.0
6	0	0	1	0	0	0	0	0	0	3700.0

将表 5.3 与上表 5.2 进行比较,发现遗传算法求解得到的总成本最小,因此,可以确信遗传算法求解此模型较优.

2. 灵敏度分析

由于题中表 1 的各个参数会发生变化,而这些变化可能会对最优决策方案产生影响.因此我们使用灵敏度分析帮助企业了解这些参数变化对决策方案的影响程度,从而在实际操作中做出更灵活和稳健的决策.

分别对零配件 1 和零配件 2 的次品率、不合格成品的调换损失、每个不合格成品的拆解成本进行灵敏度分析.

(1) 同时改变零配件 1 和零配件 2 的次品率 p_1 和 p_2 ,得到的具体决策方案及最低成本如下表 5.4 所示:

表 5.4 改变次品率结果表

p_1 和 p_2 的取值	D_1	D_2	D_f	D_d	D_r	D_{c1}	D_{c2}	D_{r1}	D_{r2}	C_{total}
0.05	0	0	1	0	0	0	0	0	0	3403.0
0.1	0	0	1	0	0	0	0	0	0	3403.0
0.2	0	0	1	0	0	0	0	0	0	3403.0
0.3	0	0	1	0	0	0	0	0	0	3403.0
0.4	0	0	1	0	0	0	0	0	0	3403.0
0.5	0	0	1	0	0	0	0	0	0	3403.0

观察到这种结果可能表明模型中检测成品 $D_f=1$ 已经提供了足够的质量控制,即使零配件次品率增加,模型依然能够通过检测成品来避免次品流入市场,从而使得总成本不变.由于检测成品的操作能有效控制次品率的影响,模型的优化解已经具有较强的稳健性,说明当前的最优策略能够有效减少总成本,并能适



应较大的次品率变化。

(2) 只改变不合格成品的调换损失 c_r ，得到的具体决策方案及最低成本如下表 5.5 所示：

表 5.5 改变不合格成品的调换损失结果表

c_r 的取值	D_1	D_2	D_f	D_d	D_r	D_{c1}	D_{c2}	D_{c1}	D_{c2}	C_{total}
6	0	0	1	0	0	0	0	0	0	3403.0
10	0	0	1	0	0	0	0	0	0	3403.0
20	0	0	1	0	0	0	0	0	0	3403.0
30	0	0	1	0	0	0	0	0	0	3403.0

观察到调换损失影响较小，这也可能表明模型中的检测成品策略 $D_f = 1$ 已经有效地减少了次品的产生，因此调换损失并不显著，以及可能是检测优先于调换损失，模型中对成品的检测操作，意味着在次品率较高的情况下，检测可以在进入市场前消除次品，从而避免调换损失的发生，因此，调换损失成本的变化对总成本和决策的影响较弱。

(3) 只改变每个不合格成品的拆解成本 c_d ，得到的具体决策方案及最低成本如下表 5.6 所示：

表 5.6 改变每个不合格成品的拆解成本结果表

c_d 的取值	D_1	D_2	D_f	D_d	D_r	D_{c1}	D_{c2}	D_{c1}	D_{c2}	C_{total}
5	0	0	1	0	0	0	0	0	0	3403.0
40	0	0	1	0	0	0	0	0	0	3403.0

模型中的拆解相关决策变量 D_d 和 D_r 一直为 0，这表明模型在当前最优策略下并不选择进行拆解，因此，拆解费用的变化不会影响总成本。

可以知道次品率、调换损失和拆解费用对总成本的影响甚微，得出以下几个结论：

(1) 模型稳健性高：当前的决策变量设置（尤其是成品检测 $D_f = 1$ ）使得模型在面对次品率、调换损失和拆解费用变化时表现出高度的稳健性，能够保持较低的总成本。

(2) 优化解已足够灵活：在不同条件下，模型总能保持相同的最优解，这说明模型中的参数设定和决策已经足够灵活，能够有效应对这些变量的波动。

(3) 次品率和相关成本影响有限：尽管我们调整了多个参数，这些变化对总成本的影响并不显著，表明这些因素在模型中的权重较低，未来可以通过调整次品率范围、降低检测比例、增加未检测成品进入市场的可能性等方式，进一步研究这些变量的影响。

5.2.5 小结

通过建立成本最小优化模型并应用遗传算法，我们确定了最优生产决策组合，实现了总成本最小化。使用遍历算法进行检验，结果证实了遗传算法的优越性，而灵敏度分析表明次品率、调换损失和拆解费用对总成本的影响较小。

5.3 问题三的模型建立与求解

在问题二的基础上，本模型进一步复杂化，引入了额外的工序和多款零配件，以更全面地模拟实际生产流程。针对两个特定的工序，分析了各自的成本构成，



并确定了五个关键的决策点：是否对零配件和成品进行检测，以及是否对不合格品进行拆解等。目标是在决策变量的指导下，实现总生产成本的最优化。满足成品的次品率、决策变量具体取值 2 个约束条件的情况下，建立了多工序成本最小化优化模型，采用混合整数线性规划算法（MILP）求解模型。

5.3.1 模型建立

本题依旧对总生产成本进行单目标优化，但更为复杂，增加了生产流程的工序，为 2 道工序、8 个零配件的情况。

5.3.1.1 决策变量的确定

根据题目要求，要使生产成本尽可能低，需要 5 个决策变量：

1. 是否对零配件 i 进行检测，假设零配件 i 为 D_i 。
2. 是否对半成品进行检测，假设半成品为 D_h 。
3. 是否对成品进行检测，假设成品为 D_f 。

上述三个决策变量的取值为 $D_i \in \{0,1\}$ ($i=1,2,f$)。当取值为 0 时，表示对其不进行检测；当取值为 1 时，表示对其进行检测。

4. 是否对不合格成品进行拆解，假设不合格成品为 D_d ，取值为 $D_d \in \{0,1\}$ 。当取值为 0 时，表示对其不进行拆解；当取值为 1 时，表示对其进行拆解。

5. 是否对退回的成品进行拆解和重新处理，假设退回的成品为 D_r ，取值为 $D_r \in \{0,1\}$ 。当取值为 0 时，表示直接丢弃不合格成品；当取值为 1 时，表示对其进行拆解。

5.3.1.2 目标函数分析

对题中给出的 2 道工序、8 个零配件的情况进行考虑，第一道工序为零配件装配成半成品，第二道工序为半成品装配成成品。

1. 第一道工序

第一道工序总成本包含零配件采购成本、检测成本、装配成本和拆解成本。其中零配件采购成本只在每个零配件进入第一道工序时计算一次，之后的工序不再重新计算零配件采购成本，只考虑装配、检测和拆解等成本。

(1) 零配件采购成本

与问题二同理，零配件采购成本 C_{bwy} 为：

$$C_{bwy} = N \sum_{i=1}^n c_{bi} \quad (5-34)$$

(2) 检测成本

零配件 i ($i=1,2,3$) 装配成半成品 1 时的检测成本 C_{dai} 为：

$$C_{dai} = N \sum_{i=1}^3 D_i c_a + D_{hi} c_{ah} + D_i (1-p_i) N \sum_{i=1}^3 D_{di} D_e p_d c_d D_i (1-D_i) + (1-D_i) N \left(\sum_{i=1}^3 D_{di} (1-(1-p_i)(1-p_i)) c_d D_i (1-D_i) \right) \quad (5-35)$$

其中， n_i 表示每件零配件的检测次数。

零配件 i ($i=4,5,6$) 装配成半成品 2 时的检测成本 C_{dai} 为：



$$C_{\text{det}1} = N \sum_{i=1}^6 D_i c_{di} + D_{\text{q}} c_{\text{dq}} + D_i (1-p_i) N \sum_{i=1}^6 D_{\text{q}} D_e p_{\text{q}} c_{di} D_{di} (1-D_i) \\ + (1-D_i) N \left(\sum_{i=1}^6 D_{\text{q}} (1-(1-p_i')(1-p_i)) c_{di} D_{di} (1-D_i) \right) \quad (5-36)$$

零配件 i ($i=7,8$) 装配成半成品 3 时的检测成本 $C_{\text{det}3}$ 为:

$$C_{\text{det}3} = N \sum_{i=1}^6 D_i c_{di} + D_{\text{q}} c_{\text{dq}} + D_i (1-p_i) N \sum_{i=1}^6 D_{\text{q}} D_e p_{\text{q}} c_{di} D_{di} (1-D_i) \\ + (1-D_i) N \left(\sum_{i=1}^6 D_{\text{q}} (1-(1-p_i')(1-p_i)) c_{di} D_{di} (1-D_i) \right) \quad (5-37)$$

因此, 总检测成本 C_{det} 为:

$$C_{\text{det}} = C_{\text{det}1} + C_{\text{det}2} + C_{\text{det}3} \quad (5-38)$$

(3) 装配成本

在多工序系统中, 零配件、半成品和成品都可能经过检测。因此, 为了避免多次对同一件物品进行成本的重复计算, 在建立模型时, 明确每个工序的检测对象。

对于初始零配件, 只有在进入第一道工序时需要计算检测成本; 对于半成品, 只有在每道工序结束时进行检测; 对于成品, 则只在最终工序结束时考虑检测。

零配件 i ($i=1,2,3$) 装配成半成品 1 时的装配成本 C_1 为:

$$C_1 = D_i (1-p_i) N \left[\sum_{i=1}^3 D_{\text{q}} D_e p_{\text{q}} (1-D_i) \frac{c_{di}}{2} \right] \quad (5-39)$$

零配件 i ($i=4,5,6$) 装配成半成品 2 时的装配成本 C_2 为:

$$C_2 = D_i (1-p_i) N \left[\sum_{i=1}^6 D_{\text{q}} D_e p_{\text{q}} D_{di} (1-p_i) \frac{c_{di}}{2} (1-D_i) \right] \quad (5-40)$$

零配件 i ($i=7,8$) 装配成半成品 3 时的装配成本 C_3 为:

$$C_3 = D_i (1-p_i) N \left[\sum_{i=1}^8 (1-D_{\text{q}}) D_e p_{\text{q}} (1-D_i) \frac{c_{di}}{2} \right] \quad (5-41)$$

因此, 零配件装配成半成品的总装配成本为:

$$C_{\text{as}} = N c_{\text{e}} + C_1 + C_2 + C_3 \quad (5-42)$$

(4) 拆解成本

将半成品 1 拆解为零配件 i ($i=1,2,3$) 的拆解成本 $C_{\text{dis}1}$ 为:

$$C_{\text{dis}1} = D_i (1-p_i) N c_{di} (D_e + D_i) p_{\text{q}} + (1-D_i) N c_{di} (D_e + D_i) [1-(1-p_{\text{q}}')(1-p_{\text{q}})] \quad (5-43)$$

将半成品 2 拆解为零配件 i ($i=4,5,6$) 的拆解成本 $C_{\text{dis}2}$ 为:

$$C_{\text{dis}2} = D_i (1-p_i) N c_{di} (D_e + D_i) p_{\text{q}} + (1-D_i) N c_{di} (D_e + D_i) [1-(1-p_{\text{q}}')(1-p_{\text{q}})] \quad (5-44)$$

将半成品 3 拆解为零配件 i ($i=7,8$) 的拆解成本 $C_{\text{dis}3}$ 为:

$$C_{\text{dis}3} = D_i (1-p_i) N c_{di} (D_e + D_i) p_{\text{q}} + (1-D_i) N c_{di} (D_e + D_i) [1-(1-p_{\text{q}}')(1-p_{\text{q}})] \quad (5-45)$$

因此, 总的拆解成本 C_{dis} 为:

$$C_{\text{dis}} = C_{\text{dis}1} + C_{\text{dis}2} + C_{\text{dis}3} \quad (5-46)$$



综上所述，第一道工序的总成本 C_{total1} 为：

$$C_{total1} = C_{buy} + C_{det} + C_{as} + C_{re} \quad (5-47)$$

2. 第二道工序

假设半成品数量为 N ，成品数量为 N' ，则半成品数量与总数量 N 的关系为：

$$N' = [D_f N (1 - P_f) + N (1 - D_f)] \quad (5-48)$$

成品数量与总数量 N 的关系为：

$$N' = [D_w N' (1 - P_f) + N' (1 - D_w)] \quad (5-49)$$

(1) 检测成本

半成品的检测成本 C_{det}' 为：

$$\begin{aligned} C_{det}' = & N D_f c_{det} + D_w (1 - p_f) N \left[\sum_{i=1}^3 D_f D_w p_f c_{det} D_{wi} (1 - D_{wi}) \right] \\ & + (1 - D_w) N \left(\sum_{i=1}^3 D_f (1 - (1 - p_f) (1 - p_f)) c_{det} D_{wi} (1 - D_{wi}) \right) \\ & + D_f (1 - p_f) N \sum_{i=1}^3 (1 - D_f) D_f p_f c_{det} (1 - D_f) D_{wi} \\ & + (1 - D_w) N \sum_{i=1}^3 (1 - D_f) D_f (1 - (1 - p_f) (1 - p_f)) c_{det} (1 - D_f) D_{wi} \end{aligned} \quad (5-50)$$

(2) 装配成本

半成品装配成成品的装配成本为：

首先考虑对半成品进行检测的情况，对成品进行拆解后，直接对半成品进行装配的成本 C_{as1} 为：

$$C_{as1} = D_f (1 - p_h) N \left[\sum_{i=1}^3 D_f D_w p_{hf} (1 - D_{wi}) \frac{c_a}{2} \right] \quad (5-51)$$

拆解成品后，对拆解得到的半成品进行检测，再装配的成本 C_{as2} 为：

$$C_{as2} = D_f (1 - p_h) N \left[\sum_{i=1}^3 D_f D_w p_{hf} D_{wi} (1 - p_h) \frac{c_a}{2} (1 - D_f) \right] \quad (5-52)$$

对调换退回的成品进行拆解成半成品后，不进行检测，直接装配的成本 C_{as3} 为：

$$C_{as3} = D_f (1 - p_h) N \left[\sum_{i=1}^3 (1 - D_f) D_f p_{hf} (1 - D_{wi}) \frac{c_a}{2} \right] \quad (5-53)$$

对调换退回的成品进行拆解并进行检测，合格半成品再进行装配的成本 C_{as4} 为：

$$C_{as4} = D_f (1 - p_h) N \left[\sum_{i=1}^3 (1 - D_f) D_f p_{hf} D_{wi} (1 - p_h) \frac{c_a}{2} (1 - D_f) \right] \quad (5-54)$$

其次考虑不对半成品进行检测的情况，拆解后直接进行装配的成本 C_{as5} 为：

$$C_{as5} = (1 - D_f) N \left[\sum_{i=1}^3 D_f D_w [1 - (1 - p_{hf}) (1 - p_{hf})] (1 - D_{wi}) \frac{c_a}{2} \right] \quad (5-55)$$

拆解后对半成品进行检测后再装配的成本 C_{as6} 为：



$$C_{\text{as6}} = (1 - D_i)N \left[\sum_{j=1}^3 D_f D_d [1 - (1 - p_{N'}) (1 - p_{N'})] D_n (1 - p_h) \frac{c_a}{2} (1 - D_i) \right] \quad (5-56)$$

对调换退回的成品进行拆解后直接装配的成本 C_{as7} 为:

$$C_{\text{as7}} = (1 - D_i)N \left[\sum_{j=1}^3 (1 - D_f) D_r [1 - (1 - p_{f'}) (1 - p_{f'})] (1 - D_n) \frac{c_a}{2} \right] \quad (5-57)$$

对调换退回的成品进行拆解后, 再进行检测, 检测到的合格半成品进行装配的成本 C_{as8} 为:

$$C_{\text{as8}} = (1 - D_i)N \left[\sum_{j=1}^3 (1 - D_q) D_r [1 - (1 - p_{q'}) (1 - p_{q'})] D_n (1 - p_q) \frac{c_a}{2} (1 - D_i) \right] \quad (5-58)$$

那么, 总的装配成本 C_{as} 为:

$$C_{\text{as}} = Nc_a + C_{\text{as1}} + C_{\text{as2}} + C_{\text{as3}} + C_{\text{as4}} + C_{\text{as5}} + C_{\text{as6}} + C_{\text{as7}} + C_{\text{as8}} \quad (5-59)$$

式中, Nc_a 表示初始装配费用。

(3) 拆解成本

总的拆解成本 C_{ds} 为:

$$C_{\text{ds}} = D_i (1 - p_q) N c_r (D_d + D_r) p_{q'} + (1 - D_i) N c_r (D_d + D_r) [1 - (1 - p_{q'}) (1 - p_{q'})] \quad (5-60)$$

式中, 对于 $D_i (1 - p_q) N c_r (D_d + D_r) p_{q'}$ 表示检测后的不合格成品的拆解费用, $(1 - D_i) N c_r (D_d + D_r) [1 - (1 - p_{q'}) (1 - p_{q'})]$ 表示调换退回成品的拆解费用。

(4) 调换成本

调换损失只在半成品到成品阶段进行考虑, 因为第一道工序的半成品不流入市场。

则总的调换损失成本为:

$$C_{\text{ex}} = D_i (1 - p_h) N p_{N'} [(1 - D_f) (c_i + s)] + (1 - D_i) N [1 - (1 - p_{N'}) (1 - p_{N'})] [(1 - D_f) (c_i + s)] \quad (5-61)$$

综上所述, 第二道工序的总成本 C_{total2} 为:

$$C_{\text{total2}} = C_{\text{as7}} + C_{\text{as8}} + C_{\text{ds}} + C_{\text{ex}} \quad (5-62)$$

结合上述两道工序的成本, 使总成本最小化, 因此, 本文确立如下 1 个优化目标:

$$\min C_{\text{total}} = C_{\text{total1}} + C_{\text{total2}} \quad (5-63)$$

5.3.1.3 约束条件

分析题意可知, 存在 2 个约束条件, 分别为:

成品次品率需满足约束条件:

$$p_f \leq p_{\max} \quad (5-64)$$

决策变量的取值满足:

$$D_i, D_h, D_f, D_d, D_r \in \{0, 1\} \quad (5-65)$$

综上所述, 可以建立多工序成本最小优化模型:

$$\min C_{\text{total}} = C_{\text{total1}} + C_{\text{total2}} \quad (5-66)$$



$$s.t. \begin{cases} p_f \leq p_{\max} \\ D_i, D_h, D_f, D_d, D_r \in \{0,1\} \end{cases}$$

5.3.2 模型求解

由于问题中的决策变量（是否进行检测、是否拆解等）是离散的，只能选择“是”或“否”，因此，适合用整数变量表示；而问题中的成本与决策之间存在线性关系；且涉及多个阶段的决策和存在多个约束条件，故，我们选择使用混合整数线性规划算法（MILP）求解模型，此算法能够处理这种多层次、结构化的决策问题，混合整数线性规划算法要求决策变量包含整数变量，并且目标函数和约束条件都是线性的。

使用混合整数线性规划算法求解过程如下：

1. 创建参数列表，其中包含多个情况，每个情况由一组参数构成，此参数列表表中已给出。

2. 初始化一个空列表，用于存放每种情况的优化结果。

3. 遍历列表中的每个情况：

（1）建立最小化问题，即总成本最小，目标是找出最小化总成本的决策变量组合。

（2）定义 8 个决策变量。

（3）根据（1）参数列表中不同情况的参数，分别计算 2 道工序的成本。

（4）将上述各个成本相加，得到总成本表达式，并将其设置为问题的目标函数。

（5）对问题进行求解，找到一组决策变量的值，使得总成本最小。

（6）从求解结果中提取决策变量的值和最小总成本。

（7）将提取的决策变量值和总成本作为一个结果记录，并将其添加到结果列表中。

4. 遍历结果列表，最终可以得到每种情况的决策变量值和对应的最小总成本。

5.3.3 问题的结论

使用混合整数线性规划算法（MILP）求解得到：

对于第一道工序，当决策变量取值均为 0 时，零配件 i ($i=1,2,\dots,8$) 装配成半成品的成本最小，其中，零配件 i ($i=1,2,3$) 装配成半成品 1 的成本为 1000；零配件 i ($i=4,5,6$) 装配成半成品 2 的成本为 1000；零配件 i ($i=7,8$) 装配成半成品 3 的成本为 3600。

对于第二道工序，当半成品 D_h ($i=1,2,3$) 决策变量取值为 1，其它决策变量取值为 0 时，半成品 i ($i=1,2,3$) 装配成成品的成本最小，为 1944.0。

则对于整个工序，最小总成本为 7544.0。

5.3.4 小结

在问题二基础上，模型加入更多工序和零配件，深入模拟生产流程，针对关键工序，分析成本并设定五个决策点，目标是优化总成本，在成品次品率和决策变量约束下，构建多工序成本最小化模型，并用 MILP 算法求解。



5.4 问题四的模型建立与求解

通过构建多约束的成本最小化模型,综合考虑了次品率、成品次品率、检测成本等因素,实现了成本与质量控制的平衡。灵敏度分析显示,成品次品率对总成本影响显著,合理控制检测频次能够有效降低生产成本。通过模拟退火算法求解,模型成功找到了最优解,实现了总成本的最小化,同时保证了产品质量。

5.4.1 模型建立

本题结合问题一、问题二、问题三,对以生产总成本最小为目标的优化模型,在前面基础上增加更多个约束条件来提升模型的准确性。

据第一问的抽样样本量 n 为:

$$n = \frac{p_0(1-p_0)Z^2}{(\hat{p}-p_0)^2} = \frac{p_0(1-p_0)Z^2}{d^2} \quad (5-67)$$

计算出每个生产阶段的最小样本量 n , 从而确定检测频次。根据这些检测次数的计算结果, 嵌入问题二和问题三中的检测成本计算公式。

对于第二问的检测成本模型进行更新:

$$\begin{aligned} C_{da} = & [D_1 n(1-p_1) + (1-D_1)n] D_1 c_a + D_1(1-p_1)n \left(\sum_{i=1}^3 D_i D_d P_i C_a D_n (1-D_i) \right. \\ & \left. + (1-D_i) n \left(\sum_{j=1}^3 (D_j \times [1-(1-p_j')(1-p_j)]) C_a D_n (1-D_i) \right) \right. \\ & \left. + D_i(1-p_i)n \left(\sum_{j=1}^3 (1-D_j) D_j P_j C_a D_n (1-D_i) D_j \right) \right. \\ & \left. + (1-D_i) n \sum_{j=1}^3 (1-D_j) D_j \times [1-(1-p_j')(1-p_j)] C_a D_n (1-D_i) \right) \end{aligned} \quad (5-68)$$

式(5-68)中检测成本只与 n 挂钩, 而其他的成本项(如购买、装配、拆解、调换损失等)都与 N 挂钩。检测次数 n 是根据抽样统计学确定的, 用来估计总体中的次品率。

对第三问的检测成本模型进行更新:

$$\begin{aligned} C_{da1} = & n \sum_{i=1}^3 D_i c_{a1} + D_{hf} c_{hf} + D_1(1-p_1)n \sum_{i=1}^3 D_{hf} D_d P_i C_a D_n (1-D_i) \\ & + (1-D_1)n \left(\sum_{i=1}^3 D_{hf} [1-(1-p_i')(1-p_i)] C_a D_n (1-D_i) \right) \\ C_{da2} = & n \sum_{i=1}^6 D_i c_a + D_{hf} c_{hf} + D_1(1-p_1)n \sum_{i=1}^6 D_{hf} D_d P_i C_a D_n (1-D_i) \\ & + (1-D_1)n \left(\sum_{i=1}^6 D_{hf} [1-(1-p_i')(1-p_i)] C_a D_n (1-D_i) \right) \\ C_{da3} = & n \sum_{i=1}^8 D_i c_{a1} + D_{hf} c_{hf} + D_1(1-p_1)n \sum_{i=1}^8 D_{hf} D_d P_i C_a D_n (1-D_i) \\ & + (1-D_1)n \left(\sum_{i=1}^8 D_{hf} [1-(1-p_i')(1-p_i)] C_a D_n (1-D_i) \right) \end{aligned} \quad (5-69)$$



$$C_{da} = [D_h \times (D_i \times n(1-p_i) + (1-D_i) \times n) \times (1-p_i) + (1-D_h)n] D_f c_{df} \\ + D_h (1-p_i) [D_i \times n \times (1-p_i) + (1-D_i) \times n] \left[\sum_{h=1}^3 D_f D_d p_f c_{da} D_{ch} (1-D_h) \right]$$

根据第二问和第三问以最小化成本为目标:

$$\min C_{total} = C_{buy} + C_{da} + C_{mz} + C_{diz} + C_{exr} \quad (5-70)$$

由于检测次数决定了每次抽样检测的精度和成本, 因此通过控制检测次数来达到检测成本和生产质量的平衡.

对其增加 8 个约束条件为:

(1) 最小化检测样本量:

$$n > \frac{p_0(1-p_0)Z_{\alpha}^2}{(\hat{p}-p_0)^2} \quad (5-71)$$

此约束保证每个阶段的检测样本量足够多, 以使得次品率估计准确.

(2) 次品率估计的精度约束:

检测次数直接影响次品率的估计精度. 因此, 可以设置约束来控制次品率的估计误差. 次品率估计的精度满足:

$$|\hat{p}-p_0| \leq d_{\max} \quad (5-72)$$

其中, $\hat{p}$ 是通过抽样检测得到的次品率的估计值, $d_{\max}$ 是允许的最大误差.

(3) 生产数量约束:

如需使生产的产品达到产业规定的合格水平, 在考虑次品率的影响下, 需要额外多购买零件数来弥补生产的不合格产品. 则生产数量满足:

$$Q(1-p_i) \geq \frac{N}{1-p_f} \quad (5-73)$$

Q 为零配件需求数量, N 为企业计划生产总合格品数量.

(4) 成品次品率约束:

成品次品率上限为:

$$p_f \leq p_{\max} \quad (5-74)$$

其中, $p_{\max}$ 是企业或市场对成品次品率的最大容忍值. 这个约束保证成品的质量能够达到客户的预期要求.

(5) 检测成本约束:

为了确保检测过程不会因为过多的抽样检测而导致成本过高, 检测成本上限应满足:

$$C_{da} \leq C_{da-\max} \quad (5-75)$$

其中 $C_{da-\max}$ 是检测成本的上限值.

(6) 装配成本约束:

装配过程和拆解过程中的成本同样影响总成本, 因此设置装配和拆解成本的上限, 控制成本的上升:

$$C_{az} \leq C_{az-\max} \quad (5-76)$$

(7) 拆解成本约束:

$$C_{diz} \leq C_{diz-\max} \quad (5-77)$$

(8) 生产时间上限:



$$T_{total} \leq T_{max} \quad (5-78)$$

其中, T_{total} 是总的生产时间, T_{max} 是时间上限, 此约束确保生产流程不会因为过度检测或复杂的装配步骤而拖延时间, 影响生产效率。

$$\min C_{total} = C_{buy} + C_{da} + C_{m3} + C_{d12} + C_{exr} \quad (5-79)$$

即对第二问中的模型进行更新:

综上所述, 可以建立多约束成本最小优化模型:

$$\begin{aligned} \min C_{total} &= C_{buy} + C_{da} + C_{m3} + C_{d12} + C_{exr} \\ s.t. &\begin{cases} n > \frac{p_0(1-p_0)Z_\alpha^2}{(\hat{p}-p_0)^2} \\ |\hat{p}-p_0| \leq d_{max} \\ Q(1-p_f) \geq \frac{N}{1-p_f} \\ p_f \leq p_{max} \\ C_{da} \leq C_{da-max} \\ C_{m3} \leq C_{m3-max} \\ C_{d12} \leq C_{d12-max} \\ T_{total} \leq T_{max} \end{cases} \end{aligned} \quad (5-80)$$

5.4.2 模型求解

由于已知值是通过抽样检测方法得到的, 因此, 对于问题二和问题三的优化目标函数, 可以知道检测成本改变, 其它成本保持不变。

假设在 95% 置信水平下, 则标准正态分布的临界值为 1.645; $d=0.5$, 标称值 p_0 为 10%, 将具体数值使用代入法代入上式 5-14, 得到 n 的值, 再将其代入上式。

为了有效地处理问题的组合性、非线性和复杂性, 同时避免局部最优, 并且在存在不确定性的情况下仍能进行有效的搜索, 选择使用模拟退火算法对模型进行求解。

通过模拟退火算法, 可以找到一组决策变量的配置, 使得总生产成本最小化, 同时满足给定的约束条件。

算法求解步骤如下:

1. 初始化: 随机生成一个初始解, 设置初始温度 $T=1000$ 和冷却率 0.99。
2. 定义目标函数: 目标函数为最小化总成本。
3. 定义邻域结构: 改变某一个决策变量的状态(0 或 1)。

4. 模拟退火循环: 在当前温度下, 重复以下步骤直到达到稳定状态: 从当前解的邻域中选择一个新的解; 计算新解的目标函数值, 并与当前解比较; 如果新解更好, 则接受; 如果更差, 则以一定的概率接受。减少温度 T , 重复直到温度降到足够低。

5.4.3 问题的结论

通过使用模拟退火算法重新求解得到问题二的结果如下表 5.7 所示:



表 5.7 模拟退火算法求解的不同情况决策变量值和对应的最小总成本表

情况	D_1	D_2	D_f	D_d	D_r	D_{e1}	D_{e2}	D_{r1}	D_{r2}	C_{total}
1	1	1	0	0	0	0	0	1	0	3845.2
2	1	1	0	0	0	1	0	1	0	4654.4
3	1	1	0	0	0	0	1	1	0	4277.2
4	1	1	0	0	0	0	0	0	1	5422.4
5	1	1	0	0	0	0	0	1	1	3917.2
6	1	1	0	0	0	0	1	0	1	3385.6

对于 6 种情况，零配件 1 和零配件 2 都需要进行检测，情况 1 还需要对退回成品拆解后的零配件 1 进行检测，才能使总成本最小，为 3845.2；情况 2 还需要对拆解后的和退回成品拆解后的零配件 1 进行检测，才能使总成本最小，为 4654.4；情况 3 还需要对拆解后的零配件 2 和退回成品拆解后的零配件 1 进行检测，才能使总成本最小，为 4277.2；情况 4 还需要对退回成品拆解后的零配件 2 进行检测，才能使总成本最小，为 5422.4；情况 5 还需要对退回成品拆解后的零配件 1 和 2 进行检测，才能使总成本最小，为 3917.2；情况 6 需要对拆解后的零配件 2、退回成品拆解后的零配件 2 进行检测，才能使总成本最小，为 3385.6。

同理，求解得到问题三的结果：

对于第一道工序，当决策变量取值均为 0 时，零配件 i ($i=1,2,\dots,8$) 装配成半成品的成本最小，其中，零配件 i ($i=1,2,3$) 装配成半成品 1 的成本为 1000；零配件 i ($i=4,5,6$) 装配成半成品 2 的成本为 1000；零配件 i ($i=7,8$) 装配成半成品 3 的成本为 3600。

对于第二道工序，当半成品 1 和半成品 3 进行检验时，即 D_i ($i=1,3$) 决策变量取值为 1，其它决策变量取值为 0 时，半成品 i ($i=1,2,3$) 装配成成品的成本最小，为 972.0。

则对于整个工序，最小总成本为 6572.0。

5.4.4 检验分析

1. 遍历算法检验

由于遍历算法能够考察所有可能解，确保找到全局最优解。因此，我们使用遍历算法作为一个基准来检验模拟退火算法找到的解的质量。

使用遍历算法求解得到的问题三的结果如下表 5.8 所示：

表 5.8 遍历算法求解的不同情况决策变量值和对应的最小总成本表

情况	D_1	D_2	D_f	D_d	D_r	D_{e1}	D_{e2}	D_{r1}	D_{r2}	C_{total}
1	1	1	0	0	0	0	0	0	0	3845.2
2	1	1	0	0	0	0	0	0	0	4654.4
3	1	1	0	0	0	0	0	0	0	4277.2
4	1	1	0	0	0	0	0	0	0	5422.4
5	1	1	0	0	0	0	0	0	0	3917.2
6	1	1	0	0	0	0	0	0	0	3385.6

使用遍历算法求解得到的问题三的结果为：

对于第一道工序，当决策变量取值均为 0 时，成本最小，其中，零配件 i ($i=1,2,3$) 装配成半成品 1 的成本为 1000；零配件 i ($i=4,5,6$) 装配成半成品 2 的成本为 1000；零配件 i ($i=7,8$) 装配成半成品 3 的成本为 3600。

对于第二道工序，当半成品 1 和半成品 3 进行检验时，即 D_i ($i=1,3$) 决策变



量取值为 1，其它决策变量取值为 0 时，半成品 $i(i=1,2,3)$ 装配成成品的成本最小，为 972.0，则对于整个工序，最小总成本为 6572.0。

观察对比结果值，使用遍历算法进行检验的结果值相同，这意味着模拟退火算法找到了一个与全局最优解非常接近的解，或者在某种程度上已经找到了全局最优解。

2. 灵敏度分析

以情况 1 为例，正常情况下，成本为 3845.2。

同时改变 p_1 、 p_2 的值，当次品率 p_1 和 p_2 同时从 0.05 增加到 0.5 时，得到结果值如下表 5.9 所示：

表 5.9 零配件次品率灵敏度分析结果表

p_1, p_2	C_{total}
0.05	3936.6
0.1	3845.2
0.2	3662.4
0.3	3479.6
0.4	3296.8
0.5	3114.0

据上表 5.9 可知，总成本 C_{total} 从 3936.6 降至 3114.0，模型中的检测成本 C_{det} 与次品率相关，因为较低的次品率需要进行更频繁的检测，以保证成品质量，这使得检测成本在次品率较低时较高，随着次品率的增加，检测需求减少，从而降低检测成本，因此总成本下降。

改变 P_f 的值，得到结果值如下表 5.10 所示：

表 5.10 零配件次品率灵敏度分析结果表

P_f	C_{total}
0.05	3287.2
0.1	3845.2
0.2	4961.2
0.3	5472.6
0.4	5974.7
0.5	6477.0

随着 P_f 的增加，总成本 C_{total} 显著上升，当成品次品率从 0.05 增加到 0.5 时，总成本从 3287.2 增加到 6477.0，这表明成品次品率对总成本具有极高的敏感度。

因为成品次品率直接影响调换损失和拆解成本，次品率 P_f 较高时，更多的不合格成品流入市场，导致更多的退货和调换操作，这些操作增加了额外的售后成本，并且还可能影响品牌声誉，进一步增加了隐性成本。从数据来看，成品次品率的增加对总成本的影响远比零配件次品率显著，这强调了在生产过程中，成品次品率控制的必要性。通过在生产流程中加强对成品质量的检测，可以有效减少后续的售后服务和调换损失，从而大幅降低总成本。

5.4.5 小结

在问题四中，构建了一个多约束条件的生产成本最小化优化模型，结合问题一的检测次数计算，更新了问题二和问题三中的检测成本公式，并新增了多个约束条件，如次品率、生产数量、检测成本等，确保模型的准确性。通过模拟退火算法进行求解，找到了不同决策变量组合下的最优解，成功实现了总生产成本的



最小化。随后使用遍历算法验证,结果表明模拟退火算法能够找到与全局最优解接近的解,验证了其有效性。该模型优化了生产过程中的检测频次和次品率,最终实现了总成本的有效控制。

六、模型评价与推广

6.1 模型的评价

- 1.检测次数模型通过优化检测次数,可以在保证产品质量的同时减少不必要的检测,从而降低检测成本。
- 2.成本最小优化模型可以显著降低生产的成本,为企业带来更高的收益。
- 3.多工序成本最小优化模型可以帮助企业在生产过程中降低成本、提高效率。

6.2 模型的推广

- 1.检测次数模型可以应用于对食品安全、药品生产进行质量控制。
- 2.将检测次数模型与供应链管理、库存控制、生产调度等其他模型相结合,实现全方位的成本控制和质量管控。
- 3.多工序成本最小优化模型可以应用于电子组装、汽车制造等,来优化生产线的各个工序。

参考文献

- [1]刘凯,廖忠芳,杨波.假设检验在住户调查质量控制中的应用[J].调研世界,2015,(07):52-54.DOI:10.13778/j.cnki.11-3705/c.2015.07.012.
- [2]张晓琴,王昭元,李宇.单侧假设检验中统计量值对检验结论的影响[J].大学数学,2020,36(06):93-96.
- [3]王建康.二项分布双边收尾概率和假设检验统计量的几处修正[J].作物学报,2024,50(06):1261-1372.
- [4]李鸿.二项分布的参数估计问题研究[J].应用数学学报,2010,33(03):385-394.
- [5]张华初,楚鹏飞,谢观霞.统计分布和中心极限定理的随机模拟[J].统计与决策,2021,37(04):69-72.DOI:10.13546/j.cnki.tjyc.2021.04.015.
- [6]Albertus ,Mickael.Asymptotic Z and chi-squared tests with auxiliary information[J].Metrika,2022,85(7):1-25.
- [7]李佩杰,万海涛,赵晓慧,等.定制化求解机组组合混合整数线性规划模型的固定——推断法[J].电力系统保护与控制,2023,51(02):11-21.DOI:10.19783/j.cnki.pspc.220548.
- [8]Albertus ,Mickael.Asymptotic Z and chi-squared tests with auxiliary information[J].Metrika,2022,85(7):1-25.
- [9]王军虎.统计检验中假设的设置方法[J].统计与决策,2022,38(21):57-59.DOI:10.13546/j.cnki.tjyc.2022.21.011.
- [10]陈元文,顾胜利,刘佳.基于内共生机制遗传算法的巡回维修路径规划研究[J].军事运筹与评估,2024,39(03):48-54.DOI:10.19949/j.ans.mora.20231109.01.



附 录

1. 支撑材料

支撑材料表

名称	内容
Question1_DrawNormalDistribution.py	问题一正态分布代码
Question1_n.py	问题一样本量的代码
Question1_Sensitivity_P.py	问题一次品率灵敏度代码
Question1_Sensitivity_P0.py	问题一标称值灵敏度代码
Question1_Sensitivity_Z.py	问题一置信区间灵敏度代码
Question2_Circu.py	问题二循环遍历代码
Question2_GA.py	问题二遗传算法代码
Question2_Sen_Cl.py	问题二调换损失灵敏度代码
Question2_Sen_Ct.py	问题二拆解费用灵敏度代码
Question2_Sen_p1-p2.py	问题二次品率灵敏度代码
Question2_St1.py	问题二第1种情况成本代码
Question3_Half_1-2-3.py	问题三半成品 123 代码
Question3_Half_1-2-3_MILP.py	问题三半成品 123 (MILP) 代码
Question3_Parts_1-2-3.py	问题三零配件 123 代码
Question3_Parts_4-5-6.py	问题三零配件 456 代码
Question3_Parts_7-8.py	问题三零配件 78 代码
Question4_1.py	问题四对问题二求解代码
Question4_1_imit.py	问题四对问题二求解 (模拟退火) 代码
Question4_1_Sen_p1-p2.py	问题四对次品率灵敏度代码
Question4_1_Sen_pf.py	问题四对 pf 灵敏度代码
Question4_1_Half_1-2-3.py	问题四半成品 123 代码
Question4_1_parts_1-2-3.py	问题四零配件 123 代码
Question4_1_parts_4-5-6.py	问题四零配件 456 代码
Question4_1_parts_7-8.py	问题四零配件 78 代码

2. 代码

Question1_DrawNormalDistribution.py:

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import norm
```

设置 matplotlib 显示中文所需的字体

plt.rcParams['font.sans-serif'] = ['SimSun'] # 宋体

plt.rcParams['font.family'] = 'sans-serif' # 使用 sans-serif 字体族

plt.rcParams['font.size'] = 17



```
plt.rcParams[axes.unicode_minus] = False # 解决保存图像是负号'-'显示为方块的问题
```

```
# 创建一个数值范围，从 -3.5 到 3.5，步长为 0.1
```

```
x = np.arange(-3.5, 3.5, 0.1)
```

```
# 计算这些数值对应的正态分布概率密度
```

```
y = norm.pdf(x)
```

```
# 绘制标准正态分布曲线
```

```
plt.figure(figsize=(10, 6))
```

```
plt.plot(x, y, label="标准正态分布")
```

```
plt.fill_between(x, y, alpha=0.2)
```

```
plt.xlabel("z-score")
```

```
plt.ylabel("概率密度")
```

```
plt.legend()
```

```
plt.grid(True)
```

```
plt.show()
```

```
# 创建一个数值范围，从 -3.5 到 3.5，步长为 0.1
```

```
x = np.arange(-3.5, 3.5, 0.1)
```

```
# 计算这些数值对应的正态分布累积概率
```

```
y = norm.cdf(x)
```

```
# 找到累积概率为90%和95%的z值
```

```
z_90 = norm.ppf(0.90)
```

```
z_95 = norm.ppf(0.95)
```

```
# 绘制标准正态分布累积概率曲线
```

```
plt.figure(figsize=(10, 6))
```

```
plt.plot(x, y, label="标准正态密度分布")
```

```
plt.xlabel("z-score")
```




```

plt.ylabel("累积概率")

# 标记累积概率为90%和95%的点
plt.plot(z_90, 0.90, 'ro') # 红色点表示90%
plt.plot(z_95, 0.95, 'bo') # 蓝色点表示95%
plt.text(z_90, 0.90, f'(z={z_90:.3f},P=0.90)', verticalalignment='bottom',
horizontalalignment='right')
plt.text(z_95, 0.95, f'(z={z_95:.3f},P=0.95)', verticalalignment='bottom',
horizontalalignment='right')

plt.legend()
plt.grid(True)
plt.show()

"""
以下求出具体的值
"""

# 给定的累积概率
cumulative_probability_1 = 0.1
cumulative_probability_2 = 0.95

# 灵敏度分析
cumulative_probability_3 = 0.90
cumulative_probability_4 = 0.99
z_value3 = norm.ppf(cumulative_probability_3)
z_value4 = norm.ppf(cumulative_probability_4)

# 使用 ppf 函数（百分点函数）来找到对应的 z 值
z_value1 = norm.ppf(cumulative_probability_1)
z_value2 = norm.ppf(cumulative_probability_2)

print(z_value1, z_value2, z_value3, z_value4)

```

Question1_n.py



```

# 定义参数
p0 = 0.10 # 标称次品率
p = 0.15 # 抽样次品率
z = 1.645 # 拒收临界值

# 计算 n 的值
n = (p0 * (1 - p0) * z**2) / ((p - p0)**2)
print(n)

# 定义参数
p0 = 0.10 # 标称次品率
p = 0.15 # 抽样次品率
z = -1.282 # 拒收临界值

# 计算 n 的值
n = (p0 * (1 - p0) * z**2) / ((p - p0)**2)
print(n)

Question1_Sensitivity_P.py
import numpy as np
import matplotlib.pyplot as plt

# 设置 matplotlib 显示中文所需的字体
plt.rcParams['font.sans-serif'] = ['SimSun'] # 宋体
plt.rcParams['font.family'] = 'sans-serif' # 使用 sans-serif 字体族
plt.rcParams['font.size'] = 17
plt.rcParams['axes.unicode_minus'] = False # 解决保存图像是负号 '-' 显示为方块的问题

# 定义参数
p0 = 0.10 # 标称次品率
z = 1.645 # 拒收临界值

# 避免 p 等于 p0 的情况
p1 = np.linspace(0, 0.05, 500) # p 的范围从 0 到 0.09 (不包括 0.1)

```




```
p2 = np.linspace(0.15, 0.3, 500) # p 的范围从 0.11 到 0.3 (不包括 0.1)
```

```
# 重新定义公式
```

```
n1 = (p0 * (1 - p0) * z**2) / ((p1 - p0)**2)
```

```
n2 = (p0 * (1 - p0) * z**2) / ((p2 - p0)**2)
```

```
# 重新绘制图形
```

```
plt.figure(figsize=(10, 6))
```

```
plt.plot(p1, n1, label=r'$p \in [0, 0.05]$')
```

```
plt.plot(p2, n2, label=r'$p \in (0.15, 0.3]$')
```

```
plt.xlabel('次品率 p')
```

```
plt.ylabel('样本量 n')
```

```
plt.legend()
```

```
plt.grid(True)
```

```
plt.show()
```

```
Question1_Sensitivity_P0.py
```

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
# 设置 matplotlib 显示中文所需的字体
```

```
plt.rcParams['font.sans-serif'] = ['SimSun'] # 宋体
```

```
plt.rcParams['font.family'] = 'sans-serif' # 使用 sans-serif 字体族
```

```
plt.rcParams['font.size'] = 17
```

```
plt.rcParams['axes.unicode_minus'] = False # 解决保存图像是负号 '-' 显示为方块的问题
```

```
# 定义参数
```

```
z = 1.645 # 拒收临界值
```

```
p = 0.11 # 抽样次品率
```

```
# 定义 p0 的两个范围
```

```
p0_range1 = np.linspace(0.05, 0.09, 500) # p0 的范围从 0.05 到 0.09
```

```
p0_range2 = np.linspace(0.13, 0.15, 500) # p0 的范围从 0.13 到 0.15
```



```
# 定义公式
n_range1 = (p0_range1 * (1 - p0_range1) * z**2) / ((p - p0_range1)**2)
n_range2 = (p0_range2 * (1 - p0_range2) * z**2) / ((p - p0_range2)**2)
```

```
# 绘制图形
plt.figure(figsize=(10, 6))
plt.plot(p0_range1, n_range1, label=r'$p_0$ \in [0.05, 0.09]$')
plt.plot(p0_range2, n_range2, label=r'$p_0$ \in [0.13, 0.15]$')
plt.xlabel('标称次品率 p0')
plt.ylabel('样本量 n')
plt.legend()
plt.grid(True)
plt.show()
```

Question1_Sensitivity_Z.py

```
import numpy as np
import matplotlib.pyplot as plt
```

```
# 设置 matplotlib 显示中文所需的字体
```

```
plt.rcParams['font.sans-serif'] = ['SimSun'] # 宋体
```

```
plt.rcParams['font.family'] = 'sans-serif' # 使用 sans-serif 字体族
```

```
plt.rcParams['font.size'] = 17
```

```
plt.rcParams['axes.unicode_minus'] = False # 解决保存图像是负号 '-' 显示为方块的问题
```

```
# 定义参数
```

```
p0 = 0.10 # 标称次品率
```

```
d = 0.05 # 检测误差
```

```
# 定义 z 的取值范围
```

```
z = np.linspace(1.282, 2.326, 400)
```

```
# 计算 n 的值
```

```
n = p0 * (1 - p0) * z**2 / d**2
```




```

# 绘制 n 随 z 变化的曲线
plt.figure(figsize=(10, 6))
plt.plot(z, n, label=r'$n=\frac{p_0(1-p_0)z^2}{d^2}$')
plt.xlabel('z 值')
plt.ylabel('样本量 n')
plt.legend()
plt.grid(True)
plt.show()

```

Question2_Circu.py:

```
import itertools
```

```
# 定义参数
```

```

cases = [
    {'p1': 0.1, 'Cb1': 4, 'Cd1': 2, 'p2': 0.1, 'Cb2': 18, 'Cd2': 3, 'pf': 0.1, 'Ca': 6, 'Cdf': 3,
     'Cl': 6, 'Cr': 5, 'S': 56},
    {'p1': 0.2, 'Cb1': 4, 'Cd1': 2, 'p2': 0.2, 'Cb2': 18, 'Cd2': 3, 'pf': 0.2, 'Ca': 6, 'Cdf': 3,
     'Cl': 6, 'Cr': 5, 'S': 56},
    {'p1': 0.1, 'Cb1': 4, 'Cd1': 2, 'p2': 0.1, 'Cb2': 18, 'Cd2': 3, 'pf': 0.1, 'Ca': 6, 'Cdf': 3,
     'Cl': 30, 'Cr': 5, 'S': 26},
    {'p1': 0.2, 'Cb1': 4, 'Cd1': 1, 'p2': 0.2, 'Cb2': 18, 'Cd2': 1, 'pf': 0.2, 'Ca': 6, 'Cdf': 2,
     'Cl': 30, 'Cr': 5, 'S': 56},
    {'p1': 0.1, 'Cb1': 4, 'Cd1': 8, 'p2': 0.2, 'Cb2': 18, 'Cd2': 1, 'pf': 0.1, 'Ca': 6, 'Cdf': 2,
     'Cl': 10, 'Cr': 5, 'S': 56},
    {'p1': 0.05, 'Cb1': 4, 'Cd1': 2, 'p2': 0.05, 'Cb2': 18, 'Cd2': 3, 'pf': 0.05, 'Ca': 6, 'Cdf':
     3, 'Cl': 10, 'Cr': 40, 'S': 56}
]

```

```
N = 100
```

```
results = []
```

```
# 遍历所有决策变量组合
```

```

for D1, D2, Df, Dd, Dr1, Dr2, Dr, Dc1, Dc2 in itertools.product([0, 1], repeat=9):
    for case in cases:

```



```

N1 = (D1 * (1 - case[p1]) * N + (1 - D1) * N) + (D2 * (1 - case[p2]) * N +
(1 - D2) * N)
PF = (1 - (1 - (1 - case[p1]) * (1 - case[p2]))) * (1 - case[pf])

# 计算各项成本
C_buy = N * (case[Cb1] + case[Cb2])
C_det = (N * (D1 * case[Cd1] + D2 * case[Cd2]) + N1 * Df * case[Cdf]
+
(case[pf] * N * (D1 * (1 - case[p1]) + D2 * (1 - case[p2])) + N * (1
- PF) * ((1 - D1) * (1 - D2))) *
(Df * Dd * (case[Cd1] * (Dc1 + Dr1) * (1 - D1) + case[Cd2] * (Dc2
+ Dr2) * (1 - D2))))

C_ass = N1 * case[Ca] + N * (D1 * (1 - case[p1]) + D2 * (1 - case[p2])) * (
Df * Dd * case[pf] * case[Ca] / 2 * (2 - Dc1 - Dc2 + Dc1 * (1 - case[p1]) * (1 -
D1) + Dc2 * (1 - case[p2]) * (1 - D2)) +
(1 - Df) * Dr * case[pf] * case[Ca] / 2 * (2 - Dr1 - Dr2 + Dr1 * (1 - case[p1]) * (1 -
D1) + (1 - case[p2]) * (1 - D2)) +
N * (2 - D1 - D2) * (
Df * Dd * (1 - PF) * case[Ca] / 2 * (2 - Dc1 - Dc2 + Dc1 * (1 - case[p1]) * (1 -
D1) + Dc2 * (1 - case[p2]) * (1 - D2)) +
(1 - Df) * Dr * case[Ca] / 2 * (1 - PF) * (2 - Dr1 - Dr2 + Dr1 * (1 -
case[p1]) * (1 - D1) + Dr2 * (1 - case[p2]) * (1 - D2))
)
)
C_dis = N * case[Ct] * (Dd + Dr) * case[pf] * (D1 * (1 - case[p1]) + D2 *
(1 - case[p2])) + (1 - PF) * N * case[Ct] * (Dd + Dr) * ((1 - D1) + (1 - D2))

C_exc = N * case[pf] * ((1 - Df) * (case[Ci] + case[S])) * (D1 * (1 -
case[p1]) + D1 * (1 - case[p1])) + N * (1 - PF) * (1 - Df) * (case[Ci] + case[S]) * (2 - D1 - D2)

# 总成本
C_total = C_buy + C_det + C_ass + C_dis + C_exc

results.append({

```




```

        'case': case,
        'D1': D1,
        'D2': D2,
        'Df': Df,
        'Dd': Dd,
        'Dr1': Dr1,
        'Dr2': Dr2,
        'Dr': Dr,
        'Dc1': Dc1,
        'Dc2': Dc2,
        'C_total': C_total
    })

# 找到每种情况下的最小成本及其对应的决策变量
best_results = {}
for case in cases:
    # 使用元组作为键
    case_key = tuple(case.values())
    best_result = min([r for r in results if tuple(r[case].values()) == case_key],
key=lambda x: x['C_total'])
    best_results[case_key] = best_result

# 输出结果
for case_key, result in best_results.items():
    case_index = cases.index(dict(zip(cases[0].keys(), case_key))) + 1
    print(f"Case {case_index}:")
    print(f"    Best {result['D1']}, {result['D2']}, {result['Df']}, {result['Dd']},
{result['Dr1']}, {result['Dr2']}, {result['Dr']}, {result['Dc1']}, {result['Dc2']} Minimum
C_total: {result['C_total']}")

Question2_GA.py
import numpy as np
from deap import base, creator, tools, algorithms

# 定义遗传算法的基本参数

```



```

POPULATION_SIZE = 100 # 种群大小
P_CROSSOVER = 0.7      # 交叉概率
P_MUTATION = 0.01      # 变异概率
MAX_GENERATIONS = 50   # 最大迭代次数
HALL_OF_FAME_SIZE = 1  # 存储最优解的数量

# 定义问题的决策变量
DECISION_VARS = ['D1', 'D2', 'Df', 'Dd', 'Dr1', 'Dr2', 'Dr', 'Dc1', 'Dc2']

# 创建适配度和个体类
creator.create("FitnessMin", base.Fitness, weights=(-1.0,))
creator.create("Individual", list, fitness=creator.FitnessMin)

# 初始化工具箱
toolbox = base.Toolbox()
toolbox.register("attr_bool", np.random.choice, [0, 1], p=[0.5, 0.5])
toolbox.register("individual", tools.initRepeat, creator.Individual, toolbox.attr_bool,
n=len(DECISION_VARS))
toolbox.register("population", tools.initRepeat, list, toolbox.individual)

def evaluate(individual, case):
    # 解包个体
    D1, D2, Df, Dd, Dr1, Dr2, Dr, Dc1, Dc2 = individual

    # 根据 case 获取参数值
    p1, Cb1, Cd1, p2, Cb2, Cd2, pf, Ca, Cdf, Cl, Ct, S = cases[case]

    # 计算公式
    N = 100
    N1 = (D1 * (1 - p1) * N + (1 - D1) * N) + (D2 * (1 - p2) * N + (1 - D2) * N)
    PF = (1 - (1 - (1 - p1) * (1 - p2))) * (1 - pf)

    C_buy = N * (Cb1 + Cb2)
    C_det = (N * (D1 * Cd1 + D2 * Cd2) + Df * Cdf * N1 +

```




```

(pf * N * (D1 * (1 - p1) + D2 * (1 - p2)) + N * (1 - PF) * ((1 - D1) * (1 - D2))) *
(Df * Dd * (Cd1 * (Dc1 + Dr1) * (1 - D1) + Cd2 * (Dc2 + Dr2) * (1 - D2))))

```

```

C_ass = N1 * Ca + N * (D1 * (1 - p1) + D2 * (1 - p2)) * (
    Df * Dd * pf * Ca / 2 * (
        2 - Dc1 - Dc2 + Dc1 * (1 - p1) * (1 - D1) + Dc2 * (1 - p2) * (1 - D2)) +
    (1 - Df) * Dr * pf * Ca / 2 * (
        2 - Dr1 - Dr2 + Dr1 * (1 - p1) * (1 - D1) + (1 - p2) * (1 - D2)) +
    N * (2 - D1 - D2) * (
        Df * Dd * (1 - PF) * Ca / 2 * (
            2 - Dc1 - Dc2 + Dc1 * (1 - p1) * (1 - D1) + Dc2 * (1 - p2) * (1 - D2)) +
        (1 - Df) * Dr * Ca / 2 * (1 - PF) * (
            2 - Dr1 - Dr2 + Dr1 * (1 - p1) * (1 - D1) + Dr2 * (1 - p2) * (1 - D2))
    )
)

```

```

C_dis = N * Ct * (Dd + Dr) * pf * (D1 * (1 - p1) + D2 * (1 - p2)) + (1 - PF) * N * Ct * (Dd + Dr) * ((1 - D1) + (1 - D2))

```

```

C_exc = N * pf * ((1 - D1) * (C1 + S)) * (D1 * (1 - p1) + D1 * (1 - p1)) + N * (1 - PF) * (1 - Df) * (C1 + S) * (2 - D1 - D2)

```

```

return C_buy + C_det + C_ass + C_dis + C_exc,

```

注册评估函数

```

toolbox.register("evaluate", evaluate, case=1) # 初始设置为第一种情况

```

```

toolbox.register("select", tools.selTournament, tournsize=3)

```

```

toolbox.register("mate", tools.cxTwoPoint)

```



```
toolbox.register("mutate", tools.mutFlipBit, indpb=0.05)
```

```
cases = [  
    [0.1, 4, 2, 0.1, 18, 3, 0.1, 6, 3, 6, 5, 56],  
    [0.2, 4, 2, 0.2, 18, 3, 0.2, 6, 3, 6, 5, 56],  
    [0.1, 4, 2, 0.1, 18, 3, 0.1, 6, 3, 30, 5, 56],  
    [0.2, 4, 1, 0.2, 18, 1, 0.2, 6, 2, 30, 5, 56],  
    [0.1, 4, 8, 0.2, 18, 1, 0.1, 6, 2, 10, 5, 56],  
    [0.05, 4, 2, 0.05, 18, 3, 0.05, 6, 3, 10, 40, 56]  
]
```

```
for i, case in enumerate(cases):  
    # 更新评估函数中的 case 参数  
    toolbox.unregister("evaluate")  
    toolbox.register("evaluate", evaluate, case=i)  
  
    # 创建初始种群  
    population = toolbox.population(n=POPULATION_SIZE)  
  
    # 历史记录  
    hof = tools.HallOfFame(HALL_OF_FAME_SIZE)  
  
    # 运行遗传算法  
    algorithms.eaSimple(population, toolbox, cxpb=P_CROSSOVER,  
mutpb=P_MUTATION,  
ngen=MAX_GENERATIONS, halloffame=hof,  
verbose=False)  
  
    print(f"Case {i + 1}: Best individual is {hof[0]} with cost  
{hof[0].fitness.values[0]}")
```

Question2_Sen_Cl.py

```
import itertools
```

```
# 定义参数
```




```

cases = [
    {'p1': 0.1, 'Cb1': 4, 'Cd1': 2, 'p2': 0.1, 'Cb2': 18, 'Cd2': 3, 'pf': 0.1, 'Ca': 6, 'Cdf': 3,
     'Cl': 6, 'Cr': 5, 'S': 56},
    {'p1': 0.1, 'Cb1': 4, 'Cd1': 2, 'p2': 0.1, 'Cb2': 18, 'Cd2': 3, 'pf': 0.1, 'Ca': 6, 'Cdf': 3,
     'Cl': 10, 'Cr': 5, 'S': 56},
    {'p1': 0.1, 'Cb1': 4, 'Cd1': 2, 'p2': 0.1, 'Cb2': 18, 'Cd2': 3, 'pf': 0.1, 'Ca': 6, 'Cdf': 3,
     'Cl': 20, 'Cr': 5, 'S': 56},
    {'p1': 0.1, 'Cb1': 4, 'Cd1': 2, 'p2': 0.1, 'Cb2': 18, 'Cd2': 3, 'pf': 0.1, 'Ca': 6, 'Cdf': 3,
     'Cl': 30, 'Cr': 5, 'S': 56}
]

```

```
N = 100
```

```
results = []
```

```
# 遍历所有决策变量组合
```

```
for D1, D2, Df, Dd, Dr1, Dr2, Dr, Dc1, Dc2 in itertools.product([0, 1], repeat=9):
```

```
    for case in cases:
```

```

        N1 = (D1 * (1 - case['p1']) * N + (1 - D1) * N) + (D2 * (1 - case['p2']) * N +
        (1 - D2) * N)

```

```
        PF = (1 - (1 - (1 - case['p1']) * (1 - case['p2']))) * (1 - case['pf'])
```

```
# 计算各项成本
```

```
C_buy = N * (case['Cb1'] + case['Cb2'])
```

```

C_det = (N * (D1 * case['Cd1'] + D2 * case['Cd2']) + Df * case['Cdf'] * N1 +
        (case['pf'] * N * (D1 * (1 - case['p1']) + D2 * (1 - case['p1']))) + N * (1
        - PF) * ((1 - D1) * (1 - D2))) *
        (Df * Dd * (case['Cd1'] * (Dc1 + Dr1) * (1 - D1) + case['Cd2'] * (Dc2
        + Dr2) * (1 - D2))))

```

```

C_ass = N1 * case['Ca'] + N * (D1 * (1 - case['p1']) + D2 * (1 - case['p2'])) * (
        Df * Dd * case['pf'] * case['Ca'] / 2 * (2 - Dc1 - Dc2 + Dc1 * (1 - case['p1']) * (1 -
        D1) + Dc2 * (1 - case['p2']) * (1 - D2)) +
        (1 - Df) * Dr * case['pf'] * case['Ca'] / 2 * (2 - Dr1 - Dr2 + Dr1 * (1 - case['p1']) * (1 -
        D1) + (1 - case['p2']) * (1 - D2)) +

```



```

        N*(2-D1-D2)*(
            Df*Dd*(1-PF)*case[Ca]/2*(2-Dc1-Dc2+Dc1*(1-case[p1])*(1-
D1)+Dc2*(1-case[p2])*(1-D2))+
            (1-Df)*Dr*case[Ca]/2*(1-PF)*(2-Dr1-Dr2+Dr1*(1-
case[p1])*(1-D1)+Dr2*(1-case[p2])*(1-D2))
        )
    )
    C_dis = N * case[Ct] * (Dd + Dr) * case[pf] * (D1*(1-case[p1]) + D2 *
(1-case[p2])) + (1 - PF) * N * case[Cf] * (Dd + Dr) * ((1-D1)+(1-D2))

    C_exc = N * case[pf] * ((1-Df)*(case[Ci]+case[S])) * (D1*(1-
case[p1])+D1*(1-case[p1])) + N*(1-PF)*(1-Df)*(case[Ci]+case[S])*(2-D1-D2)

# 总成本
C_total = C_buy + C_det + C_ass + C_dis + C_exc

results.append({
    'case': case,
    'D1': D1,
    'D2': D2,
    'Df': Df,
    'Dd': Dd,
    'Dr1': Dr1,
    'Dr2': Dr2,
    'Dr': Dr,
    'Dc1': Dc1,
    'Dc2': Dc2,
    'C_total': C_total
})

```

找到每种情况下的最小成本及其对应的决策变量

```
best_results = {}
```

```
for case in cases:
```

```
    # 使用元组作为键
```

```
    case_key = tuple(case.values())
```



```

    best_result = min([r for r in results if tuple(r[case].values()) == case_key],
key=lambda x: x['C_total'])
    best_results[case_key] = best_result

```

输出结果

```

for case_key, result in best_results.items():
    case_index = cases.index(dict(zip(cases[0].keys(), case_key))) + 1
    print(f'Case {case_index}: Best {result["D1"]}, {result["D2"]}, {result["Df"]},
{result["Dd"]},
{result["Dr1"]}, {result["Dr2"]}, {result["Dr"]}, {result["Dc1"]}, {result["Dc2"]} Minimum
C_total: {result["C_total"]}')

```

Question2_Sen_Ct.py

import itertools

定义参数

```

cases = [
    {'p1': 0.1, 'Cb1': 4, 'Cd1': 2, 'p2': 0.1, 'Cb2': 18, 'Cd2': 3, 'pf': 0.1, 'Ca': 6, 'Cdf': 3,
    'Cl': 6, 'Ct': 5, 'S': 56},
    {'p1': 0.1, 'Cb1': 4, 'Cd1': 2, 'p2': 0.1, 'Cb2': 18, 'Cd2': 3, 'pf': 0.1, 'Ca': 6, 'Cdf': 3,
    'Cl': 6, 'Ct': 40, 'S': 56}
]

```

N = 100

results = []

遍历所有决策变量组合

```

for D1, D2, Df, Dd, Dr1, Dr2, Dr, Dc1, Dc2 in itertools.product([0, 1], repeat=9):
    for case in cases:

```

```

        N1 = (D1 * (1 - case[p1]) * N + (1 - D1) * N) + (D2 * (1 - case[p2]) * N +
(1 - D2) * N)

```

```

        PF = (1 - (1 - (1 - case[p1]) * (1 - case[p2]))) * (1 - case[pf])

```

计算各项成本




```

C_buy = N * (case['Cb1'] + case['Cb2'])
C_det = (N * (D1 * case['Cd1'] + D2 * case['Cd2']) + Df * case['Cdf'] * N1 +
        (case['pf'] * N * (D1*(1-case['p1']) + D2*(1-case['p1']))) + N * (1
        - PF)*((1-D1)*(1-D2)))) *
        (Df * Dd * (case['Cd1']*(Dc1 + Dr1)*(1-D1) + case['Cd2']*(Dc2
        + Dr2)*(1-D2))))

```

```

C_ass = N1 * case['Ca'] + N*(D1*(1-case['p1'])+D2*(1-case['p2'])) * (
        Df*Dd*case['pf']*case['Ca']/2*(2-Dc1-Dc2+Dc1*(1-case['p1'])*(1-
        D1)+Dc2*(1-case['p2'])*(1-D2))+
        (1-Df)*Dr*case['pf']*case['Ca']/2*(2-Dr1-Dr2+Dr1*(1-case['p1'])*(1-
        D1)+(1-case['p2'])*(1-D2))+
        N*(2-D1-D2)*
        (Df*Dd*(1-PF)*case['Ca']/2*(2-Dc1-Dc2+Dc1*(1-case['p1'])*(1-
        D1)+Dc2*(1-case['p2'])*(1-D2))+
        (1-Df)*Dr*case['Ca']/2*(1-PF)*(2-Dr1-Dr2+Dr1*(1-
        case['p1'])*(1-D1)+Dr2*(1-case['p2'])*(1-D2))
        )
        )

```

```

C_dis = N * case['Cd'] * (Dd + Dr) * case['pf'] * (D1*(1-case['p1']) + D2 *
(1-case['p2'])) + (1 - PF) * N * case['Cf'] * (Dd + Dr) * ((1-D1)+(1-D2))

```

```

C_exc = N * case['pf'] * ((1-Df)*(case['Cl']+case['S'])) * (D1*(1-
case['p1'])+D1*(1-case['p1'])) + N*(1-PF)*(1-Df)*(case['Cl']+case['S'])*(2-D1-D2)

```

总成本

```
C_total = C_buy + C_det + C_ass + C_dis + C_exc
```

```

results.append({
    'case': case,
    'D1': D1,
    'D2': D2,
    'Df': Df,
    'Dd': Dd,
    'Dr1': Dr1,

```



```

'Dr2': Dr2,
'Dr': Dr,
'Dc1': Dc1,
'Dc2': Dc2,
'C_total': C_total
))

```

找到每种情况下的最小成本及其对应的决策变量

```
best_results = {}
```

```
for case in cases:
```

```
    # 使用元组作为键
```

```
    case_key = tuple(case.values())
```

```
    best_result = min([r for r in results if tuple(r[case].values()) == case_key],
```

```
key=lambda x: x['C_total'])
```

```
    best_results[case_key] = best_result
```

输出结果

```
for case_key, result in best_results.items():
```

```
    case_index = cases.index(dict(zip(cases[0].keys(), case_key))) + 1
```

```
    print(f"Case {case_index}: Best {result['Df']}, {result['D2']}, {result['Dr']},  
{result['Dd']},
```

```
{result['Dr1']}, {result['Dr2']}, {result['Dr']}, {result['Dc1']}, {result['Dc2']} Minimum  
C_total: {result['C_total']}")
```

Question2_Sen_p1-p2.py

```
import itertools
```

定义参数

```
cases = [
```

```
    {'p1': 0.05, 'Cb1': 4, 'Cd1': 2, 'p2': 0.05, 'Cb2': 18, 'Cd2': 3, 'pf': 0.1, 'Ca': 6, 'Cdf':  
3, 'Cl': 6, 'Ct': 5, 'S': 56},
```

```
    {'p1': 0.1, 'Cb1': 4, 'Cd1': 2, 'p2': 0.1, 'Cb2': 18, 'Cd2': 3, 'pf': 0.1, 'Ca': 6, 'Cdf': 3,  
'Cl': 6, 'Ct': 5, 'S': 56},
```

```
    {'p1': 0.2, 'Cb1': 4, 'Cd1': 2, 'p2': 0.2, 'Cb2': 18, 'Cd2': 3, 'pf': 0.1, 'Ca': 6, 'Cdf': 3,  
'Cl': 6, 'Ct': 5, 'S': 56},
```



```

        {'p1': 0.3, 'Cb1': 4, 'Cd1': 2, 'p2': 0.3, 'Cb2': 18, 'Cd2': 3, 'pf': 0.1, 'Ca': 6, 'Cdf': 3,
        'Cl': 6, 'Cr': 5, 'S': 56},
        {'p1': 0.4, 'Cb1': 4, 'Cd1': 2, 'p2': 0.4, 'Cb2': 18, 'Cd2': 3, 'pf': 0.1, 'Ca': 6, 'Cdf': 3,
        'Cl': 6, 'Cr': 5, 'S': 56},
        {'p1': 0.5, 'Cb1': 4, 'Cd1': 2, 'p2': 0.5, 'Cb2': 18, 'Cd2': 3, 'pf': 0.1, 'Ca': 6, 'Cdf': 3,
        'Cl': 6, 'Cr': 5, 'S': 56}
    ]

```

N = 100

results = []

遍历所有决策变量组合

for D1, D2, Df, Dd, Dr1, Dr2, Dr, Dc1, Dc2 in itertools.product([0, 1], repeat=9):

for case in cases:

N1 = (D1 * (1 - case[p1]) * N + (1 - D1) * N) + (D2 * (1 - case[p2]) * N + (1 - D2) * N)

PF = (1 - (1 - (1 - case[p1]) * (1 - case[p2]))) * (1 - case[pf])

计算各项成本

C_buy = N * (case[Cb1] + case[Cb2])

C_det = (N * (D1 * case[Cd1] + D2 * case[Cd2]) + Df * case[Cdf] * N1 + (case[pf] * N * (D1 * (1 - case[p1]) + D2 * (1 - case[p1])) + N * (1 - PF) * ((1 - D1) * (1 - D2)))) * (Df * Dd * (case[Cd1] * (Dc1 + Dr1) * (1 - D1) + case[Cd2] * (Dc2 + Dr2) * (1 - D2))))

C_ass = N1 * case[Ca] + N * (D1 * (1 - case[p1]) + D2 * (1 - case[p2])) * (Df * Dd * case[pf] * case[Ca] / 2 * (2 - Dc1 - Dc2 + Dc1 * (1 - case[p1]) * (1 - D1) + Dc2 * (1 - case[p2]) * (1 - D2)) + (1 - Df) * Dr * case[pf] * case[Ca] / 2 * (2 - Dr1 - Dr2 + Dr1 * (1 - case[p1]) * (1 - D1) + (1 - case[p2]) * (1 - D2)) + N * (2 - D1 - D2) * (Df * Dd * (1 - PF) * case[Ca] / 2 * (2 - Dc1 - Dc2 + Dc1 * (1 - case[p1]) * (1 - D1) + Dc2 * (1 - case[p2]) * (1 - D2)) +




```

        (1-Df)*Dr*case[Ca]/2*(1-PF)*(2-Dr1-Dr2+Dr1*(1-
case[p1])*(1-D1)+Dr2*(1-case[p2])*(1-D2))
    )
    )
    C_dis = N * case[Ct] * (Dd + Dr) * case[pf] * (D1*(1-case[p1]) + D2 *
(1-case[p2])) + (1 - PF) * N * case[Cf] * (Dd + Dr) * ((1-D1)+(1-D2))

    C_exc = N * case[pf] * ((1-Df)*(case[Ci]+case[S])) * (D1*(1-
case[p1])+D1*(1-case[p1])) + N*(1-PF)*(1-Df)*(case[Ci]+case[S])*(2-D1-D2)

# 总成本
C_total = C_buy + C_det + C_ass + C_dis + C_exc

results.append({
    'case': case,
    'D1': D1,
    'D2': D2,
    'Df': Df,
    'Dd': Dd,
    'Dr1': Dr1,
    'Dr2': Dr2,
    'Dr': Dr,
    'Dc1': Dc1,
    'Dc2': Dc2,
    'C_total': C_total
})

# 找到每种情况下的最小成本及其对应的决策变量
best_results = {}
for case in cases:
    # 使用元组作为键
    case_key = tuple(case.values())
    best_result = min([r for r in results if tuple(r[case].values()) == case_key],
key=lambda x: x[C_total])
    best_results[case_key] = best_result

```



输出结果

```
for case_key, result in best_results.items():
    case_index = cases.index(dict(zip(cases[0].keys(), case_key))) + 1
    print(f'Case {case_index}: Best {result["D1"]}, {result["D2"]}, {result["Df"]},
{result["Dd"]},
{result["Dr1"]}, {result["Dr2"]}, {result["Dr"]}, {result["Dc1"]}, {result["Dc2"]} Minimum
C_total: {result["C_total"]}')

```

Question2_St1.py

import itertools

定义参数

```
cases = [
    {'p1': 0.1, 'Cb1': 4, 'Cd1': 2, 'p2': 0.1, 'Cb2': 18, 'Cd2': 3, 'pf': 0.1, 'Ca': 6, 'Cdf': 3,
    'Cl': 6, 'Ct': 5, 'S': 56}
]

```

N = 10

results = []

遍历所有决策变量组合

```
for D1, D2, Df, Dd, Dr1, Dr2, Dr, Dc1, Dc2 in itertools.product([0, 1], repeat=9):

```

```
    for case in cases:

```

```
        # 计算 Q1 和 Q2

```

```
        Q1 = max(1, int(N / ((1 - case["p1"]) * (1 - case["pf"]))))

```

```
        Q2 = max(1, int(N / ((1 - case["p2"]) * (1 - case["pf"]))))

```

```
        # 计算各项成本

```

```
        C_buy = N * (case["Cb1"] + case["Cb2"])

```

```
        C_det = N * (

```

```
            D1 * (case["Cd1"]) + D2 * (case["Cd2"]) +

```

```
            Df * case["Cdf"] +

```

```
            Df * Dd * case["pf"] * (case["Cd1"] * Dc1 * (1 - D1) + case["Cd2"] * Dc2

```

```
            * (1 - D2)) +

```



```

        (1 - Df) * Dr * case[pf] * (
            case[Cd1] * (1 - D1) * Dr1 + case[Cd2] * (1 - D2) * Dr2
        )
    )
    C_ass = N * (
        case[Ca] +
        Df * Dd * (1 - (1 - case[p1]) * (1 - case[p2])) * (
            (1 - Dc1) * (case[Ca] / 2) +
            (1 - Dc2) * (case[Ca] / 2) +
            Dc1 * (1 - case[p1]) * (case[Ca] / 2) * (1 - D1) +
            Dc2 * (1 - case[p2]) * (case[Ca] / 2) * (1 - D2)
        ) +
        (1 - Df) * Dr * (1 - (1 - case[p1]) * (1 - case[p2])) * (
            (1 - Dr1) * (case[Ca] / 2) +
            (1 - Dr2) * (case[Ca] / 2) +
            Dr1 * (1 - case[p1]) * (case[Ca] / 2) * (1 - D1) +
            Dr2 * (1 - case[p2]) * (case[Ca] / 2) * (1 - D2)
        )
    )
    C_dis = N * (Dd * case[pf] * case[Ct] + Dr1 * case[pf] * case[Ct] + Dr2
* case[pf] * case[Ct])
    C_exc = N * ((1 - Df) * case[pf] * (case[Ct] + case[S]))

# 总成本
C_total = C_buy + C_det + C_ass + C_dis + C_exc

results.append({
    'case': case,
    'D1': D1,
    'D2': D2,
    'Df': Df,
    'Dd': Dd,
    'Dr1': Dr1,
    'Dr2': Dr2,
    'Dr': Dr,

```




```

        'Dc1': Dc1,
        'Dc2': Dc2,
        'Q1': Q1,
        'Q2': Q2,
        'C_total': C_total
    ))

# 找到每种情况下的最小成本及其对应的决策变量
best_results = {}
for case in cases:
    # 使用元组作为键
    case_key = tuple(case.values())
    best_result = min([r for r in results if tuple(r[case].values()) == case_key],
key=lambda x: x['C_total'])
    best_results[case_key] = best_result

# 输出结果
for case_key, result in best_results.items():
    case_index = cases.index(dict(zip(cases[0].keys(), case_key))) + 1
    print(f"Case {case_index}:")
    print(f"  Best D1: {result['D1']}, D2: {result['D2']}, Df: {result['Df']}, Dd: {result['Dd']}, Dr1: {result['Dr1']}, Dr2: {result['Dr2']}, Dr: {result['Dr']}, Dc1: {result['Dc1']}, Dc2: {result['Dc2']}")
    print(f"  Best Q1: {result['Q1']}, Q2: {result['Q2']}")
    print(f"  Minimum C_total: {result['C_total']}\n")

Question3_Half_1-2-3.py
import itertools

# 定义参数
cases = [
    {'p1': 0.1, 'Cb1': 22, 'Cd1': 4, 'p2': 0.1, 'Cb2': 22, 'Cd2': 4, 'p3': 0.1, 'Cb3': 20, 'Cd3': 4, 'pf': 0.1, 'Ca': 6, 'Cdf': 6, 'Cl': 40, 'Ct': 6, 'S': 200}
]

N = 100

```



```
results = []
```

```
# 遍历所有决策变量组合
```

```
for D1, D2,D3, Df, Dd, Dr1, Dr2, Dr3, Dr, Dc1, Dc2, Dc3 in itertools.product([0, 1],
repeat=12):
```

```
    for case in cases:
```

```
        N1 = (D1 * (1 - case[p1]) * N + (1 - D1) * N) + (D2 * (1 - case[p2]) * N +
(1 - D2) * N)
```

```
        N2 = (D1 * N1 * (1 - case[p1]) + (1 - D1) * N1)
```

```
        PF = (1 - (1 - (1 - case[p1]) * (1 - case[p2]))) * (1 - case[pf])
```

```
        C_det = N2 * Df * case[Cdf] +
```

```
            D1 * (1 - case[p1]) * N1 * (case[Cd1] * Dc1 * (1 - D1) + case[Cd2] * Dc2 * (1 -
D2) + case[Cd3] * Dc3 * (1 - D3)) * (Df * Dd * case[pf] + (1 - D1) * N1 * Df * (1 - PF)) +
```

```
            (case[Cd1] * Dr1 * (1 - D1) + case[Cd2] * Dr2 * (1 -
D2) + case[Cd3] * Dr3 * (1 - D3)) * (D1 * (1 - case[p1]) * N1 * (1 - Df) * Dr * case[pf] + (1 -
D1) * N1 * (1 - Df) * Dr * (1 - PF))
```

```
        C_ass = D1 * (1 - case[p1]) * N1 * (
```

```
            case[Ca] + (Df * Dd * case[pf] * case[Ca] / 2) * (1 - Dc1) * (1 - Dc2) * (1 - Dc3)
```

```
        +
```

```
            (Df * Dd * case[pf] * case[Ca] / 2) * (Dc1 * (1 - case[p1]) * (1 - D1) + Dc2 * (1 -
case[p2]) * (1 - D2) + Dc3 * (1 - case[p3]) * (1 - D3)) +
```

```
            (1 - Df) * Dr * case[pf] * case[Ca] / 2 * (3 - Dr1 - Dr2 - Dr3) +
```

```
            (1 - Df) * Dr * case[pf] * case[Ca] / 2 * (Dr1 * (1 - D1) * (1 - case[p1]) + Dr2 * (1 -
D2) * (1 - case[p2]) + Dr3 * (1 - D3) * (1 - case[p3]))
```

```
        ) +
```

```
        (1 - D1) * N1 * (
```

```
            case[Ca] + (Df * Dd * (1 - PF) * case[Ca] / 2) * (1 - Dc1) * (1 -
Dc2) * (1 - Dc3) +
```

```
            (Df * Dd * (1 - PF) * case[Ca] / 2) * (Dc1 * (1 - case[p1]) * (1 - D1)
+ Dc2 * (1 - case[p2]) * (1 - D2) + Dc3 * (1 - case[p3]) * (1 - D3)) +
```

```
            (1 - Df) * Dr * (1 - PF) * case[Ca] / 2 * (3 - Dr1 - Dr2 - Dr3) +
```

```
            (1 - Df) * Dr * (1 - PF) * case[Ca] / 2 * (Dr1 * (1 - D1) * (1 -
```



```

case[p1]) + Dr2 * (1 - D2) * (1 - case[p2]) + Dr3 * (1 - D3) * (1 - case[p3]))
    )
    C_dis = N1*(Dd+Dr)*case[Cf]*(D1*case[pf]*(1-case[p1])+(1-D1)*(1-
PF))

    C_exc = N1 * (1 - Df)*(case[CI]+case[S])*(D1 * (1-
case[p1])*case[pf]+(1-D1)*(1-PF))
    # 总成本
    C_total = C_det + C_ass + C_dis + C_exc

    results.append({
        'case': case,
        'D1': D1,
        'D2': D2,
        'Dc3': Dc3,
        'Df': Df,
        'Dd': Dd,
        'Dr1': Dr1,
        'Dr2': Dr2,
        'Dr3': Dr3,
        'Dr': Dr,
        'Dc1': Dc1,
        'Dc2': Dc2,
        'Dc3': Dc3,
        'C_total': C_total
    })

# 找到每种情况下的最小成本及其对应的决策变量
best_results = {}
for case in cases:
    # 使用元组作为键
    case_key = tuple(case.values())
    best_result = min([r for r in results if tuple(r[case].values()) == case_key],
key=lambda x: x[C_total])
    best_results[case_key] = best_result

```



输出结果

```
for case_key, result in best_results.items():
    case_index = cases.index(dict(zip(cases[0].keys(), case_key))) + 1
    print(f"Case {case_index}:")
    print(f"    Best {result['D1']}, {result['D2']}, {result['Df']}, {result['Dd']},
{result['Dr1']}, {result['Dr2']}, {result['Dr']}, {result['Dc1']}, {result['Dc2']}    Minimum
C_total: {result['C_total']}")
```

Question3_Half_1-2-3_MILP.py

import pulp

N = 1000

定义参数列表

```
cases = [
    {'p1': 0.1, 'Cb1': 4, 'Cd1': 2, 'p2': 0.1, 'Cb2': 18, 'Cd2': 3, 'pf': 0.1, 'Ca': 6, 'Cdf': 3,
    'Cl': 6, 'Cr': 5, 'S': 56},
    {'p1': 0.2, 'Cb1': 4, 'Cd1': 2, 'p2': 0.2, 'Cb2': 18, 'Cd2': 3, 'pf': 0.2, 'Ca': 6, 'Cdf': 3,
    'Cl': 6, 'Cr': 5, 'S': 56},
    {'p1': 0.1, 'Cb1': 4, 'Cd1': 2, 'p2': 0.1, 'Cb2': 18, 'Cd2': 3, 'pf': 0.1, 'Ca': 6, 'Cdf': 3,
    'Cl': 30, 'Cr': 5, 'S': 56},
    {'p1': 0.2, 'Cb1': 4, 'Cd1': 1, 'p2': 0.2, 'Cb2': 18, 'Cd2': 1, 'pf': 0.2, 'Ca': 6, 'Cdf': 2,
    'Cl': 30, 'Cr': 5, 'S': 56},
    {'p1': 0.1, 'Cb1': 4, 'Cd1': 8, 'p2': 0.2, 'Cb2': 18, 'Cd2': 1, 'pf': 0.1, 'Ca': 6, 'Cdf': 2,
    'Cl': 10, 'Cr': 5, 'S': 56},
    {'p1': 0.05, 'Cb1': 4, 'Cd1': 2, 'p2': 0.05, 'Cb2': 18, 'Cd2': 3, 'pf': 0.05, 'Ca': 6, 'Cdf':
    3, 'Cl': 10, 'Cr': 40, 'S': 56}
]
```

初始化结果列表

results = []

遍历每种情况

for case in cases:



```

# 创建一个最小化问题
prob = pulp.LpProblem("CostMinimization", pulp.LpMinimize)

# 定义决策变量
D1 = pulp.LpVariable('D1', cat='Binary')
D2 = pulp.LpVariable('D2', cat='Binary')
Df = pulp.LpVariable('Df', cat='Binary')
Dd = pulp.LpVariable('Dd', cat='Binary')
Dr1 = pulp.LpVariable('Dr1', cat='Binary')
Dr2 = pulp.LpVariable('Dr2', cat='Binary')
Dr = pulp.LpVariable('Dr', cat='Binary')
Dc1 = pulp.LpVariable('Dc1', cat='Binary')
Dc2 = pulp.LpVariable('Dc2', cat='Binary')

# 计算 Q1 和 Q2
Q1 = pulp.LpVariable('Q1', lowBound=int(N / ((1 - case[p1]) * (1 - case[pf]))),
cat='Integer')
Q2 = pulp.LpVariable('Q2', lowBound=int(N / ((1 - case[p2]) * (1 - case[pf]))),
cat='Integer')

# 定义目标函数
C_buy = N * (case[Cb1] * Q1 + case[Cb2] * Q2)

C_det = N * (
    D1 * (case[Cd1] / (1 - case[p1])) +
    D2 * (case[Cd2] / (1 - case[p2])) +
    Df * case[Cdf] +
    Dd * case[pf] * (
        case[Cd1] * Dc1 * (1 - D1) +
        case[Cd2] * Dc2 * (1 - D2)
    ) +
    Dr1 * case[pf] * (case[Cd1] * (1 - D1)) +
    Dr2 * case[pf] * (case[Cd2] * (1 - D2))
)

```



```

C_ass = N * (
    case[Ca] +
    Dd * (1 - (1 - case[p1]) * (1 - case[p2])) * (
        (1 - Dc1) * (case[Ca] / 2) +
        (1 - Dc2) * (case[Ca] / 2) +
        Dc1 * (1 - case[p1]) * (case[Ca] / 2) * (1 - D1) +
        Dc2 * (1 - case[p2]) * (case[Ca] / 2) * (1 - D2)
    ) +
    Dr * (1 - (1 - case[p1]) * (1 - case[p2])) * (
        (1 - Dr1) * (case[Ca] / 2) +
        (1 - Dr2) * (case[Ca] / 2) +
        Dr1 * (1 - case[p1]) * (case[Ca] / 2) * (1 - D1) +
        Dr2 * (1 - case[p2]) * (case[Ca] / 2) * (1 - D2)
    )
)

C_dis = N * (Dd * case[pf] * case[Cf] + Dr1 * case[pf] * case[Cf] + Dr2 *
case[pf] * case[Cf])

C_exc = N * ((1 - Df) * case[pf] * (case[Cl] + case[S]))

prob += C_buy + C_det + C_ass + C_dis + C_exc, "TotalCost"

# 解决问题
prob.solve()

# 存储结果
result = {
    'D1': D1.varValue,
    'D2': D2.varValue,
    'Df': Df.varValue,
    'Dd': Dd.varValue,
    'Dr1': Dr1.varValue,
    'Dr2': Dr2.varValue,
    'Dr': Dr.varValue,

```




```

        'Dc1': Dc1.varValue,
        'Dc2': Dc2.varValue,
        'Q1': Q1.varValue,
        'Q2': Q2.varValue,
        'C_total': pulp.value(prob.objective)
    }

    results.append(result)

# 输出结果
for i, result in enumerate(results):
    print(f"Case {i+1}: {result}")

Question3_Parts_1-2-3.py
import itertools

# 定义参数
cases = [
    ('p1': 0.1, 'Cb1': 2, 'Cd1': 1, 'p2': 0.1, 'Cb2': 8, 'Cd2': 1, 'p3': 0.1, 'Cb3': 12, 'Cd3': 2,
    'pf': 0.1, 'Ca': 8, 'Cd4': 4, 'C': 6)
]

N = 100
results = []

# 遍历所有决策变量组合
for D1, D2, Df, Dd, Dn, D3, Dc1, Dc2, Dc3, in itertools.product([0, 1], repeat=9):
    for case in cases:
        N1 = (D1 * (1 - case[p1]) * N + (1 - D1) * N) + (D2 * (1 - case[p2]) * N +
        (1 - D2) * N)
        PF = (1 - (1 - (1 - case[p1]) * (1 - case[p2]))) * (1 - case[pf])

# 计算各项成本
C_buy = N * (case[Cb1] + case[Cb2])

```



```

C_det = (N *
          Df * case[Cdf] * (D1 * case[Cd1] + D2 * case[Cd2] + D3 *
case[Cd1]) +
          (D1*(1-case[p1])*case[pf]*(case[Cd1]*Dc1 * (1-D1)
+case[Cd2]*Dc2 * (1-D2)+case[Cd3]*Dc3 * (1-D3)))+
          ((1-D1)*(Df*(1-PF)*((case[Cd1]*Dc1*(1-
D1))+(case[Cd1]*Dc1*(1-D1))+(case[Cd1]*Dc1*(1-D1))))))
        )

C_ass = (N1*case[Ca]+D1*(1-case[p1])*N+(1-D1)*N)*(
          Df*Dd*(1-PF)*case[Ca]/2*(3-D1-D2-D3+Dc1*(1-case[p1'])*(1-
D1)+Dc2*(1-case[p2'])*(1-D2)+Dc3*(1-case[p3'])*(1-D3))
        )
C_dis = N * case[Cf] * Dd * (
          D1*(1-case[p1])*case[pf]+(1-D1)*(1-PF)
        )

# 总成本
C_total = C_buy + C_det + C_ass + C_dis

results.append({
    'case': case,
    'D1': D1,
    'D2': D2,
    'D3': D3,
    'Df': Df,
    'Dd': Dd,
    'Dr1': Dr1,
    'Dc1': Dc1,
    'Dc2': Dc2,
    'Dc3': Dc3,
    'C_total': C_total
})

```



```

# 找到每种情况下的最小成本及其对应的决策变量
best_results = {}
for case in cases:
    # 使用元组作为键
    case_key = tuple(case.values())
    best_result = min([r for r in results if tuple(r[case].values()) == case_key],
key=lambda x: x['C_total'])
    best_results[case_key] = best_result

# 输出结果
for case_key, result in best_results.items():
    case_index = cases.index(dict(zip(cases[0].keys(), case_key))) + 1
    print(f"Case 零配件 1+2+3:")
    print(f"    Best {result['D1']}, {result['D2']}, {result['Df']}, {result['Dd']},
{result['Dr1']}, {result['D3']}, {result['Dc3']}, {result['Dc1']}, {result['Dc2']}
Minimum C_total: {result['C_total']}")

Question3_Parts_45-6.py
import itertools

# 定义参数
cases = [
    {'p1': 0.1, 'Cb1': 2, 'Cd1': 1, 'p2': 0.1, 'Cb2': 8, 'Cd2': 1, 'p3': 0.1, 'Cb3': 12, 'Cd3': 2,
'pf': 0.1, 'Ca': 8, 'Cdf': 4, 'Cr': 6}
]

N = 100
results = []

# 遍历所有决策变量组合
for D1, D2, Df, Dd, Dr1, D3, Dc1, Dc2, Dc3, in itertools.product([0, 1], repeat=9):
    for case in cases:
        N1 = (D1 * (1 - case['p1']) * N + (1 - D1) * N) + (D2 * (1 - case['p2']) * N +
(1 - D2) * N)

```




```

PF = (1 - (1 - (1 - case[p1]) * (1 - case[p2]))) * (1 - case[pf])

# 计算各项成本
C_buy = N * (case[Cb1] + case[Cb2])
C_det = (N *
          Df * case[Cdf] * (D1 * case[Cd1] + D2 * case[Cd2] + D3 *
case[Cd1]) +
          (D1 * (1 - case[p1]) * case[pf] * (case[Cd1] * Dc1 * (1 - D1)
+ case[Cd2] * Dc2 * (1 - D2) + case[Cd3] * Dc3 * (1 - D3))) +
          ((1 - D1) * (Df * (1 - PF) * ((case[Cd1] * Dc1 * (1 -
D1)) + (case[Cd1] * Dc1 * (1 - D1)) + (case[Cd1] * Dc1 * (1 - D1))))))
          )

C_ass = (N1 * case[Ca] + D1 * (1 - case[p1]) * N + (1 - D1) * N) * (
          Df * Dd * (1 - PF) * case[Ca] / 2 * (3 - D1 - D2 - D3 + Dc1 * (1 - case[p1]) * (1 -
D1) + Dc2 * (1 - case[p2]) * (1 - D2) + Dc3 * (1 - case[p3]) * (1 - D3))
          )
C_dis = N * case[Cf] * Dd * (
          D1 * (1 - case[p1]) * case[pf] + (1 - D1) * (1 - PF)
          )

# 总成本
C_total = C_buy + C_det + C_ass + C_dis

results.append({
    'case': case,
    'D1': D1,
    'D2': D2,
    'D3': D3,
    'Df': Df,
    'Dd': Dd,
    'Dr1': Dr1,
    'Dc1': Dc1,
    'Dc2': Dc2,

```



```

        'Dc3': Dc3,
        'C_total': C_total
    })

# 找到每种情况下的最小成本及其对应的决策变量
best_results = {}
for case in cases:
    # 使用元组作为键
    case_key = tuple(case.values())
    best_result = min([r for r in results if tuple(r[case].values()) == case_key],
key=lambda x: x['C_total'])
    best_results[case_key] = best_result

# 输出结果
for case_key, result in best_results.items():
    case_index = cases.index(dict(zip(cases[0].keys(), case_key))) + 1
    print(f"Case 零配件 4+5+6:")
    print(f"    Best {result[D1]}, {result[D2]}, {result[Df]}, {result[Dd]},
{result[Dr1]}, {result[D3]}, {result[Dc3]}, {result[Dc1]}, {result[Dc2]}
Minimum C_total: {result['C_total']}")

Question3_Parts_7-8.py
import itertools

# 定义参数
cases = [
    {'p1': 0.1, 'Cb1': 8, 'Cd1': 1, 'p2': 0.1, 'Cb2': 12, 'Cd2': 2, 'pf': 0.1, 'Ca': 8, 'Cdf': 4,
    'Ct': 6}
]

N = 100
results = []

# 遍历所有决策变量组合
for D1, D2, Df, Dd, Dr1, Dr2, Dr, Dc1, Dc2 in itertools.product([0, 1], repeat=9):

```



```

for case in cases:

    N1 = (D1 * (1 - case[p1]) * N + (1 - D1) * N) + (D2 * (1 - case[p2]) * N +
(1 - D2) * N)
    PF = (1 - (1 - (1 - case[p1]) * (1 - case[p2]))) * (1 - case[pf])

    # 计算各项成本
    C_buy = N * (case[Cb1] + case[Cb2])
    C_det = (N * (D1 * case[Cd1] + D2 * case[Cd2]) + N1 * Df * case[Cdf]
+
        (case[pf] * N * (D1 * (1 - case[p1]) + D2 * (1 - case[p1]))) + N * (1
- PF) * ((1 - D1) * (1 - D2))) *
        (Df * Dd * (case[Cd1] * Dc1 * (1 - D1) + case[Cd2] * Dc2 * (1 - D2))))

    C_ass = N1 * case[Ca] + N * (D1 * (1 - case[p1]) + D2 * (1 - case[p2])) * (
        Df * Dd * case[pf] * case[Ca] / 2 * (2 - Dc1 - Dc2 + Dc1 * (1 - case[p1]) * (1 -
D1) + Dc2 * (1 - case[p2]) * (1 - D2)) +
        (1 - Df) * case[pf] * case[Ca] / 2 * (2 + (1 - case[p1]) * (1 - D1) + (1 -
case[p2]) * (1 - D2)) +
        N * (2 - D1 - D2) * (
            Df * Dd * (1 - PF) * case[Ca] / 2 * (2 - Dc1 - Dc2 + Dc1 * (1 - case[p1]) * (1 -
D1) + Dc2 * (1 - case[p2]) * (1 - D2)) +
            (1 - Df) * case[Ca] / 2 * (1 - PF) * (2 + (1 - case[p1]) * (1 - D1) + (1 -
case[p2]) * (1 - D2))
        )
    )
    C_dis = N * case[Ct] * (Dd) * case[pf] * (D1 * (1 - case[p1]) + D2 * (1 -
case[p2])) + (1 - PF) * N * case[Ct] * (Dd) * ((1 - D1) + (1 - D2))

    # 总成本
    C_total = C_buy + C_det + C_ass + C_dis

    results.append({
        'case': case,

```




```

        'D1': D1,
        'D2': D2,
        'Df': Df,
        'Dd': Dd,
        'Dr1': Dr1,
        'Dr2': Dr2,
        'Dr': Dr,
        'Dc1': Dc1,
        'Dc2': Dc2,
        'C_total': C_total
    ))

# 找到每种情况下的最小成本及其对应的决策变量
best_results = {}
for case in cases:
    # 使用元组作为键
    case_key = tuple(case.values())
    best_result = min([r for r in results if tuple(r[case].values()) == case_key],
key=lambda x: x['C_total'])
    best_results[case_key] = best_result

# 输出结果
for case_key, result in best_results.items():
    case_index = cases.index(dict(zip(cases[0].keys(), case_key))) + 1
    print(f"Case 零配件 7+8")
    print(f"    Best {result['D1']}, {result['D2']}, {result['Df']}, {result['Dd']},
{result['Dr1']},{result['Dr2']},{result['Dr']},{result['Dc1']},{result['Dc2']} Minimum
C_total: {result['C_total']}")

附录四：
Question4_1.py
import itertools

# 定义参数
cases = [
    ('p1': 0.1, 'Cb1': 4, 'Cd1': 2, 'p2': 0.1, 'Cb2': 18, 'Cd2': 3, 'pf': 0.1, 'Ca': 6, 'Cdf': 3,

```



```

'Cl': 6, 'Cr': 5, 'S': 56},
    {'p1': 0.2, 'Cb1': 4, 'Cd1': 2, 'p2': 0.2, 'Cb2': 18, 'Cd2': 3, 'pf': 0.2, 'Ca': 6, 'Cdf': 3,
'Cl': 6, 'Cr': 5, 'S': 56},
    {'p1': 0.1, 'Cb1': 4, 'Cd1': 2, 'p2': 0.1, 'Cb2': 18, 'Cd2': 3, 'pf': 0.1, 'Ca': 6, 'Cdf': 3,
'Cl': 30, 'Cr': 5, 'S': 56},
    {'p1': 0.2, 'Cb1': 4, 'Cd1': 1, 'p2': 0.2, 'Cb2': 18, 'Cd2': 1, 'pf': 0.2, 'Ca': 6, 'Cdf': 2,
'Cl': 30, 'Cr': 5, 'S': 56},
    {'p1': 0.1, 'Cb1': 4, 'Cd1': 8, 'p2': 0.2, 'Cb2': 18, 'Cd2': 1, 'pf': 0.1, 'Ca': 6, 'Cdf': 2,
'Cl': 10, 'Cr': 5, 'S': 56},
    {'p1': 0.05, 'Cb1': 4, 'Cd1': 2, 'p2': 0.05, 'Cb2': 18, 'Cd2': 3, 'pf': 0.05, 'Ca': 6, 'Cdf':
3, 'Cl': 10, 'Cr': 40, 'S': 56}
]

```

```

N = 100
n = 98# 59
results = []

```

遍历所有决策变量组合

```

for D1, D2, Df, Dd, Dr1, Dr2, Dr, Dc1, Dc2 in itertools.product([0, 1], repeat=9):
    for case in cases:

```

```

        N1 = (D1 * (1 - case[p1])) * n + (1 - D1) * n
        PF = (1 - (1 - (1 - case[p1]) * (1 - case[p2]))) * (1 - case[pf])

```

计算各项成本

```

        C_buy = N * (case[Cb1] + case[Cb2])
        C_det = n * N1 * Df * case[Cdf] + \
            (D1 * (1 -
case[p1]) * N) * (D * Dd * case[pf] * n * (case[Cd1] * Dc1 * (1 - D1) + (case[Cd2] * Dc2 * (1 -
D2)))) + \
            D1 * (1 - case[p1]) * N * (1 - Df) * Dr * case[pf] * (case[Cd1] * Dr1 * (1 -
D1) + case[Cd2] * Dr2 * (1 - D2)) * n + \
            (1 - D1) * N * (1 - Df) * Dr * (1 - PF) * (case[Cd1] * (1 -
D1) * Dr1 + case[Cd2] * (1 - D2) * Dr2)

```



```

C_ass = N1 * case[Ca] + N*(D1*(1-case[p1])+D2*(1-case[p2])) * (
    Df*Dd*case[pf]*case[Ca]/2*(2-Dc1-Dc2+Dc1*(1-case[p1])*(1-
D1)+Dc2*(1-case[p2])*(1-D2))+
    (1-Df)*Dr*case[pf]*case[Ca]/2*(2-Dr1-Dr2+Dr1*(1-case[p1])*(1-
D1)+(1-case[p2])*(1-D2))+
    N*(2-D1-D2)*(
        Df*Dd*(1-PF)*case[Ca]/2*(2-Dc1-Dc2+Dc1*(1-case[p1])*(1-
D1)+Dc2*(1-case[p2])*(1-D2))+
        (1-Df)*Dr*case[Ca]/2*(1-PF)*(2-Dr1-Dr2+Dr1*(1-
case[p1])*(1-D1)+Dr2*(1-case[p2])*(1-D2))
    )
)
C_dis = N * case[Ct] * (Dd + Dr) * case[pf] * (D1*(1-case[p1]) + D2 *
(1-case[p2])) + (1 - PF) * N * case[Ct] * (Dd + Dr) * ((1-D1)+(1-D2))

C_exc = N * case[pf] * ((1-Df)*(case[Cl]+case[S])) * (D1*(1-
case[p1])+D1*(1-case[p1])) + N*(1-PF)*(1-Df)*(case[Cl]+case[S])*(2-D1-D2)

# 总成本
C_total = C_buy + C_det + C_ass + C_dis + C_exc

results.append({
    'case': case,
    'D1': D1,
    'D2': D2,
    'Df': Df,
    'Dd': Dd,
    'Dr1': Dr1,
    'Dr2': Dr2,
    'Dr': Dr,
    'Dc1': Dc1,
    'Dc2': Dc2,
    'C_total': C_total
})

```




```

# 找到每种情况下的最小成本及其对应的决策变量
best_results = {}
for case in cases:
    # 使用元组作为键
    case_key = tuple(case.values())
    best_result = min([r for r in results if tuple(r[case].values()) == case_key],
key=lambda x: x['C_total'])
    best_results[case_key] = best_result

```

输出结果

```

for case_key, result in best_results.items():
    case_index = cases.index(dict(zip(cases[0].keys(), case_key))) + 1
    print(f'Case {case_index}:')
    print(f'    Best {result["D1"]}, {result["D2"]}, {result["Df"]}, {result["Dd"]},
{result["Dr1"]}, {result["Dr2"]}, {result["Dr"]}, {result["Dc1"]}, {result["Dc2"]}, Minimum
C_total: {result["C_total"]}')

```

Question4_1_init.py

```

import random
import math

```

N = 100

n = 98

定义参数

```

cases = [
    {'p1': 0.1, 'Cb1': 4, 'Cd1': 2, 'p2': 0.1, 'Cb2': 18, 'Cd2': 3, 'pf': 0.1, 'Ca': 6, 'Cdf': 3,
    'Cl': 6, 'Ct': 5, 'S': 56},
    {'p1': 0.2, 'Cb1': 4, 'Cd1': 2, 'p2': 0.2, 'Cb2': 18, 'Cd2': 3, 'pf': 0.2, 'Ca': 6, 'Cdf': 3,
    'Cl': 6, 'Ct': 5, 'S': 56},
    {'p1': 0.1, 'Cb1': 4, 'Cd1': 2, 'p2': 0.1, 'Cb2': 18, 'Cd2': 3, 'pf': 0.1, 'Ca': 6, 'Cdf': 3,
    'Cl': 30, 'Ct': 5, 'S': 56},
    {'p1': 0.2, 'Cb1': 4, 'Cd1': 1, 'p2': 0.2, 'Cb2': 18, 'Cd2': 1, 'pf': 0.2, 'Ca': 6, 'Cdf': 2,
    'Cl': 30, 'Ct': 5, 'S': 56},

```



```

        {'p1': 0.1, 'Cb1': 4, 'Cd1': 8, 'p2': 0.2, 'Cb2': 18, 'Cd2': 1, 'pf': 0.1, 'Ca': 6, 'Cdf': 2,
        'Cl': 10, 'Cr': 5, 'S': 56},
        {'p1': 0.05, 'Cb1': 4, 'Cd1': 2, 'p2': 0.05, 'Cb2': 18, 'Cd2': 3, 'pf': 0.05, 'Ca': 6, 'Cdf':
        3, 'Cl': 10, 'Cr': 40, 'S': 56}
    ]

```

```

def calculate_cost(case, D1, D2, Df, Dd, Dr1, Dr2, Dr, Dc1, Dc2):
    N1 = (D1 * (1 - case[p1]) * n + (1 - D1) * n)
    PF = (1 - (1 - (1 - case[p1]) * (1 - case[p2]))) * (1 - case[pf])

    # 计算各项成本
    C_buy = N * (case[Cb1] + case[Cb2])
    C_det = n * N1 * Df * case[Cdf] + \
        (D1 * (1 - case[p1]) * N) * (
            Df * Dd * case[pf] * n * (case[Cd1] * Dc1 * (1 - D1) +
            (case[Cd2] * Dc2 * (1 - D2)))) + \
        D1 * (1 - case[p1]) * N * (1 - Df) * Dr * case[pf] * (
            case[Cd1] * Dr1 * (1 - D1) + case[Cd2] * Dr2 * (1 -
D2)) * n + \
        (1 - D1) * N * (1 - Df) * Dr * (1 - PF) * (case[Cd1] * (1 - D1) * Dr1 +
case[Cd2] * (1 - D2) * Dr2)

    C_ass = N1 * case[Ca] + N * (D1 * (1 - case[p1]) + D2 * (1 - case[p2])) * (
        Df * Dd * case[pf] * case[Ca] / 2 * (
            2 - Dc1 - Dc2 + Dc1 * (1 - case[p1]) * (1 - D1) + Dc2 * (1 -
case[p2]) * (1 - D2)) +
        (1 - Df) * Dr * case[pf] * case[Ca] / 2 * (
            2 - Dr1 - Dr2 + Dr1 * (1 - case[p1]) * (1 - D1) + (1 -
case[p2]) * (1 - D2)) +
        N * (2 - D1 - D2) * (
            Df * Dd * (1 - PF) * case[Ca] / 2 * (
                2 - Dc1 - Dc2 + Dc1 * (1 - case[p1]) * (1 - D1) + Dc2 *
(1 - case[p2]) * (1 - D2)) +
            (1 - Df) * Dr * case[Ca] / 2 * (1 - PF) * (

```



```

                2 - Dr1 - Dr2 + Dr1 * (1 - case[p1]) * (1 - D1)
+ Dr2 * (1 - case[p2]) * (1 - D2))
            )
        )
        C_dis = N * case[Cf] * (Dd + Dr) * case[pf] * (D1 * (1 - case[p1]) + D2 * (1 -
case[p2])) + (1 - PF) * N * \
            case[Cf] * (Dd + Dr) * ((1 - D1) + (1 - D2))

        C_exc = N * case[pf] * ((1 - Df) * (case[CI] + case[S])) * (
            D1 * (1 - case[p1]) + D1 * (1 - case[p1])) + N * (1 - PF) * (1 -
Df) * (case[CI] + case[S]) * (
                2 - D1 - D2)

        # 总成本
        C_total = C_buy + C_det + C_ass + C_dis + C_exc
        return C_total

```

```

def generate_initial_solution():
    return [random.choice([0, 1]) for _ in range(9)]

def get_neighbor(solution):
    new_solution = solution.copy()
    index_to_change = random.randint(0, len(new_solution) - 1)
    new_solution[index_to_change] = 1 - new_solution[index_to_change]
    return new_solution

def acceptance_probability(cost_diff, T):
    return math.exp(-cost_diff / T)

def simulated_annealing(case):
    current_solution = generate_initial_solution()

```




```

current_cost = calculate_cost(case, *current_solution)

T = 1000 # 初始温度
alpha = 0.99 # 冷却率
while T > 1e-5:
    for _ in range(100): # 每个温度下的迭代次数
        new_solution = get_neighbor(current_solution)
        new_cost = calculate_cost(case, *new_solution)
        cost_diff = new_cost - current_cost
        if cost_diff < 0 or random.random() < acceptance_probability(cost_diff,
T):
            current_solution = new_solution
            current_cost = new_cost
        T *= alpha

    return current_solution, current_cost

# 对每个案例应用模拟退火算法
best_results = {}
for case in cases:
    case_key = tuple(case.values())
    best_solution, best_cost = simulated_annealing(case)
    best_results[case_key] = {
        'D1': best_solution[0],
        'D2': best_solution[1],
        'Df': best_solution[2],
        'Dd': best_solution[3],
        'Dr1': best_solution[4],
        'Dr2': best_solution[5],
        'Dr': best_solution[6],
        'Dc1': best_solution[7],
        'Dc2': best_solution[8],
        'C_total': best_cost
    }

```



输出结果

```
for case_key, result in best_results.items():
    case_index = cases.index(dict(zip(cases[0].keys(), case_key))) + 1
    print(f"Case {case_index}:")
    print(
        f"    Best {result['D1']}, {result['D2']}, {result['Df']}, {result['Dd']},
        {result['Dr1']}, {result['Dr2']}, {result['Dr']}, {result['Dc1']}, {result['Dc2']}    Minimum
        C_total: {result['C_total']}")
```

Question4_2_Half_1-2-3.py

import itertools

定义参数

```
cases = [
    {'p1': 0.1, 'Cb1': 22, 'Cd1': 4, 'p2': 0.1, 'Cb2': 22, 'Cd2': 4, 'p3': 0.1, 'Cb3': 20, 'Cd3':
    4, 'pf': 0.1, 'Ca': 6, 'Cdf': 6, 'Cl': 40, 'Cf': 6, 'S': 200}
]
```

N = 100

n = 98

results = []

遍历所有决策变量组合

```
for D1, D2, D3, Df, Dd, Dr1, Dr2, Dr3, Dr, Dc1, Dc2, Dc3 in itertools.product([0, 1],
repeat=12):
```

```
    for case in cases:
```

```
        N1 = (D1 * (1 - case['p1']) * N + (1 - D1) * n)
```

```
        N2 = (D1 * N1 * (1 - case['p1']) + (1 - D1) * N1)
```

```
        PF = (1 - (1 - (1 - case['p1']) * (1 - case['p2']))) * (1 - case['pf'])
```

```
        C_det = N2 * Df * case['Cdf'] + \
```

```
            D1 * (1 - case['p1']) * N1 * (case['Cd1'] * Dc1 * (1 - D1) + case['Cd2'] * Dc2 * (1 -
            D2) + case['Cd3'] * Dc3 * (1 - D3)) + (Df * Dd * case['pf'] + (1 - D1) * N1 * Df * (1 - PF)) + \
```



```

        (case[Cd1]*Dr1*(1-D1)+case[Cd2]*Dr2*(1-
D2)+case[Cd3]*Dr3*(1-D3))*(D1*(1-case[p1])*N1*(1-Df)*Dr*case[pf]+(1-
D1)*N1*(1-Df)*Dr*(1-PF))

C_ass = D1*(1-case[p1])*N1*(
        case[Ca] + (Df*Dd*case[pf]*case[Ca]/2)*(1-Dc1)*(1-Dc2)*(1-Dc3)
+
        (Df*Dd*case[pf]*case[Ca]/2)*(Dc1*(1-case[p1])*(1-D1)+Dc2*(1-
case[p2])*(1-D2)+Dc3*(1-case[p3])*(1-D3)) +
        (1-Df)*Dr*case[pf]*case[Ca]/2*(3-Dr1-Dr2-Dr3)+
        (1-Df)*Dr*case[pf]*case[Ca]/2*(Dr1*(1-D1)*(1-case[p1])+Dr2*(1-
D2)*(1-case[p2])+Dr3*(1-D3)*(1-case[p3]))
    )+1
    (1 - D1) * N1 * (
        case[Ca] + (Df * Dd * (1-PF) * case[Ca] / 2) * (1 - Dc1)*(1 -
Dc2)*(1 - Dc3) +
        (Df* Dd * (1-PF) * case[Ca] / 2)*(Dc1 * (1 - case[p1]) * (1 - D1)
+ Dc2 * (1 - case[p2]) * (1 - D2) + Dc3 * (1 - case[p3]) * (1 - D3)) +
        (1 - Df) * Dr * (1-PF) * case[Ca] / 2 * (3 - Dr1 - Dr2 - Dr3) +
        (1 - Df) * Dr * (1-PF) * case[Ca] / 2 * (Dr1 * (1 - D1) * (1 -
case[p1]) + Dr2 * (1 - D2) * (1 - case[p2]) + Dr3 * (1 - D3) * (1 - case[p3]))
    )
    C_dis = N1*(Dd+Dr)*case[Cf]*(D1*case[pf]*(1-case[p1])+(1-D1)*(1-
PF))

C_exc = N1 * (1 - Df)*(case[CI]+case[S])*(D1 * (1-
case[p1])*case[pf]+(1-D1)*(1-PF))
# 总成本
C_total=C_det+C_ass+C_dis+C_exc

results.append({
    'case': case,
    'D1': D1,
    'D2': D2,
    'D3': D3,

```




```

'Df': Df,
'Dd': Dd,
'Dr1': Dr1,
'Dr2': Dr2,
'Dr3': Dr3,
'Dr': Dr,
'Dc1': Dc1,
'Dc2': Dc2,
'Dc3': Dc3,
'C_total': C_total
))

# 找到每种情况下的最小成本及其对应的决策变量
best_results = {}
for case in cases:
    # 使用元组作为键
    case_key = tuple(case.values())
    best_result = min([r for r in results if tuple(r[case.values()]) == case_key],
key=lambda x: x['C_total'])
    best_results[case_key] = best_result

# 输出结果
for case_key, result in best_results.items():
    case_index = cases.index(dict(zip(cases[0].keys(), case_key))) + 1
    print(f"Case {case_index}:")
    print(f"    Best {result['D1']}, {result['D2']}, {result['Df']}, {result['Dd']},
{result['Dr1']}, {result['Dr2']}, {result['Dr']}, {result['Dc1']}, {result['Dc2']} Minimum
C_total: {result['C_total']}")

Question4_2_Parts_1-2-3.py
import itertools

# 定义参数
cases = [
    ('p1': 0.1, 'Cb1': 2, 'Cd1': 1, 'p2': 0.1, 'Cb2': 8, 'Cd2': 1, 'p3': 0.1, 'Cb3': 12, 'Cd3': 2,

```



```
'pf': 0.1, 'Ca': 8, 'Cdf': 4, 'Ct': 6}
]
```

```
N = 100
```

```
n = 98
```

```
results = []
```

```
# 遍历所有决策变量组合
```

```
for D1, D2, Df, Dd, Dr1, D3, Dc1, Dc2, Dc3, in itertools.product([0, 1], repeat=9):
```

```
    for case in cases:
```

```
        N1 = (D1 * (1 - case[p1]) * N + (1 - D1) * N) + (D2 * (1 - case[p2]) * N +
(1 - D2) * N)
```

```
        PF = (1 - (1 - (1 - case[p1]) * (1 - case[p2]))) * (1 - case[pf])
```

```
        # 计算各项成本
```

```
        C_buy = N * (case[Cb1] + case[Cb2])
```

```
        C_det = (n *
                Df * case[Cdf] * (D1 * case[Cd1] + D2 * case[Cd2] + D3 *
case[Cd1]) +
                (D1 * (1 - case[p1]) * case[pf] * (case[Cd1] * Dc1 * (1 - D1)
+ case[Cd2] * Dc2 * (1 - D2) + case[Cd3] * Dc3 * (1 - D3))) +
                ((1 - D1) * (Df * (1 - PF) * ((case[Cd1] * Dc1 * (1 -
D1)) + (case[Cd1] * Dc1 * (1 - D1)) + (case[Cd1] * Dc1 * (1 - D1)))))
                )
```

```
        C_ass = (N1 * case[Ca] + D1 * (1 - case[p1]) * N + (1 - D1) * N) *
                (Df * Dd * (1 - PF) * case[Ca] / 2 * (3 - D1 - D2 - D3 + Dc1 * (1 - case[p1]) * (1 -
D1) + Dc2 * (1 - case[p2]) * (1 - D2) + Dc3 * (1 - case[p3]) * (1 - D3))
                )
```

```
        C_dis = N * case[Ct] * Dd * (
                D1 * (1 - case[p1]) * case[pf] + (1 - D1) * (1 - PF)
                )
```



```

# 总成本
C_total = C_buy + C_det + C_ass + C_dis

results.append({
    'case': case,
    'D1': D1,
    'D2': D2,
    'D3': D3,
    'Df': Df,
    'Dd': Dd,
    'Dr1': Dr1,
    'Dc1': Dc1,
    'Dc2': Dc2,
    'Dc3': Dc3,
    'C_total': C_total
})

# 找到每种情况下的最小成本及其对应的决策变量
best_results = {}
for case in cases:
    # 使用元组作为键
    case_key = tuple(case.values())
    best_result = min([r for r in results if tuple(r[case].values()) == case_key],
        key=lambda x: x['C_total'])
    best_results[case_key] = best_result

# 输出结果
for case_key, result in best_results.items():
    case_index = cases.index(dict(zip(cases[0].keys(), case_key))) + 1
    print(f"Case 零配件 1+2+3:")
    print(f"    Best {result['D1']}, {result['D2']}, {result['Df']}, {result['Dd']},
{result['Dr1']},{result['D3']},{result['Dc3']},{result['Dc1']},{result['Dc2']}
Minimum C_total: {result['C_total']}")

```

Question4_2_Parts_4-5-6.py




```

import itertools

# 定义参数
cases = [
    {'p1': 0.1, 'Cb1': 2, 'Cd1': 1, 'p2': 0.1, 'Cb2': 8, 'Cd2': 1, 'p3': 0.1, 'Cb3': 12, 'Cd3': 2,
    'pf': 0.1, 'Ca': 8, 'Cdf': 4, 'Ct': 6}
]

N = 100
n = 98
results = []

# 遍历所有决策变量组合
for D1, D2, Df, Dd, Dr1, D3, Dc1, Dc2, Dc3, in itertools.product([0, 1], repeat=9):
    for case in cases:

        N1 = (D1 * (1 - case['p1']) * N + (1 - D1) * N) + (D2 * (1 - case['p2']) * N +
        (1 - D2) * N)
        PF = (1 - (1 - (1 - case['p1']) * (1 - case['p2']))) * (1 - case['pf'])

        # 计算各项成本
        C_buy = N * (case['Cb1'] + case['Cb2'])
        C_det = (n *
            Df * case['Cdf'] * (Dd * case['Cd1'] + D2 * case['Cd2'] + D3 *
            case['Cd1']) +
            (D1 * (1 - case['p1']) * case['pf'] * (case['Cd1'] * Dc1 * (1 - D1)
            + case['Cd2'] * Dc2 * (1 - D2) + case['Cd3'] * Dc3 * (1 - D3))) +
            ((1 - D1) * (Df * (1 - PF) * ((case['Cd1'] * Dc1 * (1 -
            D1)) + (case['Cd1'] * Dc1 * (1 - D1)) + (case['Cd1'] * Dc1 * (1 - D1))))))

        C_ass = (N1 * case['Ca'] + D1 * (1 - case['p1']) * N + (1 - D1) * N) * (
            Df * Dd * (1 - PF) * case['Ca'] / 2 * (3 - D1 - D2 - D3 + Dc1 * (1 - case['p1']) * (1 -
            D1) + Dc2 * (1 - case['p2']) * (1 - D2) + Dc3 * (1 - case['p3']) * (1 - D3))
            )

```



```

C_dis = N * case[Cf] * Dd * (
    D1*(1-case[p1])*case[pf]+(1-D1)*(1-PF)
)

# 总成本
C_total = C_buy + C_det + C_ass + C_dis

results.append({
    'case': case,
    'D1': D1,
    'D2': D2,
    'D3': D3,
    'Df': Df,
    'Dd': Dd,
    'Dr1': Dr1,
    'Dc1': Dc1,
    'Dc2': Dc2,
    'Dc3': Dc3,
    'C_total': C_total
})

# 找到每种情况下的最小成本及其对应的决策变量
best_results = {}
for case in cases:
    # 使用元组作为键
    case_key = tuple(case.values())
    best_result = min([r for r in results if tuple(r[case].values()) == case_key],
        key=lambda x: x['C_total'])
    best_results[case_key] = best_result

# 输出结果
for case_key, result in best_results.items():
    case_index = cases.index(dict(zip(cases[0].keys(), case_key))) + 1
    print(f"Case 零配件 4+5+6:")

```



```

    print(f"    Best {result[D1]}, {result[D2]}, {result[Df]}, {result[Dd]},
{result[Dr1]}, {result[D3]}, {result[Dc3]}, {result[Dc1]}, {result[Dc2]}
Minimum C_total: {result[C_total]}")

```

Question4_2_Parts_7-8.py

```
import itertools
```

```
# 定义参数
```

```

cases = [
    {'p1': 0.1, 'Cb1': 8, 'Cd1': 1, 'p2': 0.1, 'Cb2': 12, 'Cd2': 2, 'pf': 0.1, 'Ca': 8, 'Cdf': 4,
    'Ct': 6}
]

```

```
N = 100
```

```
n = 98
```

```
results = []
```

```
# 遍历所有决策变量组合
```

```
for D1, D2, Df, Dd, Dr1, Dr2, D3, Dc1, Dc2 in itertools.product([0, 1], repeat=9):
```

```
    for case in cases:
```

```

        N1 = (D1 * (1 - case[p1]) * N + (1 - D1) * N) + (D2 * (1 - case[p2]) * N +
(1 - D2) * N)

```

```
        PF = (1 - (1 - (1 - case[p1]) * (1 - case[p2]))) * (1 - case[pf])
```

```
# 计算各项成本
```

```
C_buy = N * (case[Cb1] + case[Cb2])
```

```

C_det = (n * (D1 * case[Cd1] + D2 * case[Cd2]) + N1 * Df * case[Cdf] +
(case[pf] * n * (D1 * (1 - case[p1]) + D2 * (1 - case[p1])) + n * (1 -
PF) * ((1 - D1) * (1 - D2)))) *

```

```
    (Df * Dd * (case[Cd1] * Dc1 * (1 - D1) + case[Cd2] * Dc2 * (1 - D2)))
```

```
C_ass = N1 * case[Ca] + N * (D1 * (1 - case[p1]) + D2 * (1 - case[p2])) * (
```

```

    Df * Dd * case[pf] * case[Ca] / 2 * (2 - Dc1 - Dc2 + Dc1 * (1 - case[p1]) * (1 -
D1) + Dc2 * (1 - case[p2]) * (1 - D2)) +

```




```

        (1-Df)*case[pf]*case[Ca]/2*(2+(1-case[p1])*(1-D1)+(1-
case[p2])*(1-D2))+
        N*(2-D1-D2)*(
            Df*Dd*(1-PF)*case[Ca]/2*(2-Dc1-Dc2+Dc1*(1-case[p1])*(1-
D1)+Dc2*(1-case[p2])*(1-D2))+
            (1-Df)*case[Ca]/2*(1-PF)*(2+(1-case[p1])*(1-D1)+(1-
case[p2])*(1-D2))
        )
    )
    C_dis = N * case[Ct] * (Dd) * case[pf] * (D1*(1-case[p1]) + D2 * (1-
case[p2])) + (1 - PF) * N * case[Ct] * (Dd) * ((1-D1)+(1-D2))

```

总成本

C_total = C_buy + C_det + C_ass + C_dis

```

results.append({
    'case': case,
    'D1': D1,
    'D2': D2,
    'Df': Df,
    'Dd': Dd,
    'Dr1': Dr1,
    'Dr2': Dr2,
    'Dr': Dr,
    'Dc1': Dc1,
    'Dc2': Dc2,
    'C_total': C_total
})

```

找到每种情况下的最小成本及其对应的决策变量

best_results = {}

for case in cases:

使用元组作为键

case_key = tuple(case.values())



```

    best_result = min([r for r in results if tuple(r[case].values()) == case_key],
key=lambda x: x[C_total])
    best_results[case_key] = best_result

# 输出结果
for case_key, result in best_results.items():
    case_index = cases.index(dict(zip(cases[0].keys(), case_key))) + 1
    print(f"Case 零配件 7+8:")
    print(f"    Best {result[D1]}, {result[D2]}, {result[Df]}, {result[Dd]},
{result[Dr1]}, {result[Dr2]}, {result[Dr]}, {result[Dc1]}, {result[Dc2]}    Minimum
C_total: {result[C_total]}")

```

Cherzing
Cherzing@qq.com

